

Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking

Jinkun Cao¹ Jiangmiao Pang² Xinshuo Weng³ Rawal Khirodkar¹ Kris Kitani¹
¹ Carnegie Mellon University ² Shanghai AI Laboratory ³Nvidia

Abstract

Kalman filter (KF) based methods for multi-object tracking (MOT) make an assumption that objects move linearly. While this assumption is acceptable for very short periods of occlusion, linear estimates of motion for prolonged time can be highly inaccurate. Moreover, when there is no measurement available to update Kalman filter parameters, the standard convention is to trust the priori state estimations for posteriori update. This leads to the accumulation of errors during a period of occlusion. The error causes significant motion direction variance in practice. In this work, we show that a basic Kalman filter can still obtain state-of-the-art tracking performance if proper care is taken to fix the noise accumulated during occlusion. Instead of relying only on the linear state estimate (i.e., estimation-centric approach), we use object observations (i.e., the measurements by object detector) to compute a virtual trajectory over the occlusion period to fix the error accumulation of filter parameters during the occlusion period. This allows more time steps to correct errors accumulated during occlusion. We name our method **Observation-Centric SORT (OC-SORT)**. It remains Simple, Online, and Real-Time but improves robustness during occlusion and non-linear motion. Given off-the-shelf detections as input, OC-SORT runs at 700+ FPS on a single CPU. It achieves state-of-the-art on multiple datasets, including MOT17, MOT20, KITTI, head tracking, and especially DanceTrack where the object motion is highly non-linear. The code and models are available at https://github.com/noahcao/OC_SORT.

1. Introduction

We aim to develop a motion model-based multi-object tracking (MOT) method that is robust to occlusion and non-linear motion. Most existing motion model-based algorithms assume that the tracking targets have a constant velocity within a time interval, which is called the linear motion assumption. This assumption breaks in many practical scenarios, but it still works because when the time interval

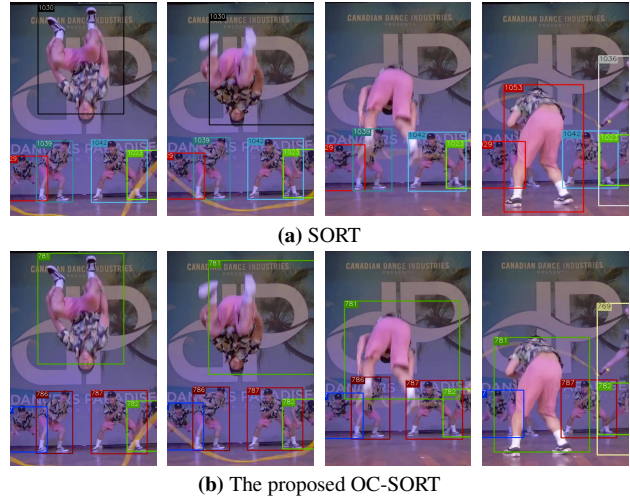


Figure 1. Samples from the results on DanceTrack [54]. SORT and OC-SORT use the same detection results. On the third frame, SORT encounters an ID switch for the backflip target while ours not.

is small enough, the object’s motion can be reasonably approximated as linear. In this work, we are motivated by the fact that most of the errors from motion model-based tracking methods occur when occlusion and non-linear motion happen together. To mitigate the adverse effects caused, we first rethink current motion models and recognize some limitations. Then, we propose addressing them for more robust tracking performance, especially in occlusion.

As the main branch of motion model-based tracking, filtering-based methods assume a transition function to predict the state of objects on future time steps, which are called state “estimations”. Besides estimations, they leverage an observation model, such as an object detector, to derive the state measurements of target objects, also called “observations”. Observations usually serve as auxiliary information to help update the posteriori parameters of the filter. The trajectories are still extended by the state estimations. Among this line of work, the most widely used one is SORT [3], which uses a Kalman filter (KF) to estimate

object states and a linear motion function as the transition function between time steps. However, SORT shows insufficient tracking robustness when the object motion is non-linear, and no observations are available when updating the filter posteriori parameters.

In this work, we recognize **three limitations** of SORT. First, although the high frame rate is the key to approximating the object motion as linear, it also amplifies the model’s sensitivity to the noise of state estimations. Specifically, between consecutive frames of a high frame-rate video, we demonstrate that the noise of displacement of the object can be of the same magnitude as the actual object displacement, leading to the estimated object velocity by KF suffering from a significant variance. Also, the noise in the velocity estimate will accumulate into the position estimate by the transition process. Second, the noise of state estimations by KF is accumulated along the time when there is no observation available in the update stage of KF. We show that the error accumulates very fast with respect to the time of the target object’s being untracked. The noise’s influence on the velocity direction often makes the track lost again even after re-association. Last, given the development of modern detectors, the object state by detections usually has lower variance than the state estimations propagated along time steps by a fixed transition function in filters. However, SORT is designed to prolong the object trajectories by state estimations instead of observations.

To relieve the negative effect of these limitations, we propose two main innovations in this work. First, we design a module to use object state observations to reduce the accumulated error during the track’s being lost in a backcheck fashion. To be precise, besides the traditional stages of *predict* and *update*, we add a stage of *re-update* to correct the accumulated error. The *re-update* is triggered when a track is re-activated by associating to an observation after a period of being untracked. The *re-update* uses virtual observations on the historical time steps to prevent error accumulation. The virtual observations come from a trajectory generated using the last-seen observation before untracked and the latest observation re-activating this track as anchors. We name it *Observation-centric Re-Update (ORU)*.

Besides ORU, the assumption of linear motion provides the consistency of the object motion direction. But this cue is hard to be used in SORT’s association because of the heavy noise in direction estimation. But we propose an observation-centric manner to incorporate the direction consistency of tracks in the cost matrix for the association. We name it *Observation-Centric Momentum (OCM)*. We also provide analytical justification for the noise of velocity direction estimation in practice.

The proposed method, named as **Observation-Centric SORT** or **OC-SORT** in short, remains simple, online, real-time and significantly improves robustness over occlusion

and non-linear motion. Our contributions are summarized as the following:

1. We recognize, analytically and empirically, three limitations of SORT, *i.e.* sensitivity to the noise of state estimations, error accumulation over time, and being estimation-centric;
2. We propose OC-SORT for tracking under occlusion and non-linear motion by fixing SORT’s limitations. It achieves state-of-the-art performance on multiple datasets in an online and real-time fashion.

2. Related Works

Motion Models. Many modern MOT algorithms [3, 11, 63, 70, 73] use motion models. Typically, these motion models use Bayesian estimation [34] to predict the next state by maximizing a posterior estimation. As one of the most classic motion models, Kalman filter (KF) [30] is a recursive Bayes filter that follows a typical predict-update cycle. The true state is assumed to be an unobserved Markov process, and the measurements are observations from a hidden Markov model [44]. Given that the linear motion assumption limits KF, follow-up works like Extended KF [52] and Unscented KF [28] were proposed to handle non-linear motion with first-order and third-order Taylor approximation. However, they still rely on approximating the Gaussian prior assumed by KF and require motion pattern assumption. On the other hand, particle filters [22] solve the non-linear motion by sampling-based posterior estimation but require exponential order of computation. Therefore, these variants of Kalman filter and particle filters are rarely adopted in the visual multi-object tracking and the mostly adopted motion model is still based on Kalman filter [3].

Multi-object Tracking. As a classic computer vision task, visual multi-object tracking is traditionally approached from probabilistic perspectives, *e.g.* joint probabilistic association [1]. And modern video object tracking is usually built upon modern object detectors [46, 48, 74]. SORT [3] adopts the Kalman filter for motion-based multi-object tracking given observations from deep detectors. DeepSORT [63] further introduces deep visual features [23, 51] into object association under the framework of SORT. Re-identification-based object association [42, 63, 71] has also become popular since then but falls short when scenes are crowded and objects are represented coarsely (*e.g.* enclosed by bounding boxes), or object appearance is not distinguishable. More recently, transformers [58] have been introduced to MOT [8, 39, 55, 69] to learn deep representations from both visual information and object trajectories. However, their performance still has a significant gap between state-of-the-art tracking-by-detection methods in terms of both accuracy and time efficiency.

3. Rethink the Limitations of SORT

In this section, we review Kalman filter and SORT [3]. We recognize some of their limitations, which are significant with **occlusion** and **non-linear object motion**. We are motivated to improve tracking robustness by fixing them.

3.1. Preliminaries

Kalman filter (KF) [30] is a linear estimator for dynamical systems discretized in the time domain. KF only requires the state estimations on the previous time step and the current measurement to estimate the target state on the next time step. The filter maintains two variables, the posteriori state estimate $\mathbf{x}$, and the posteriori estimate covariance matrix $\mathbf{P}$. In the task of object tracking, we describe the KF process with the state transition model $\mathbf{F}$, the observation model $\mathbf{H}$, the process noise $\mathbf{Q}$, and the observation noise $\mathbf{R}$. At each step t , given observations $\mathbf{z}_t$, KF works in an alternation of *predict* and *update* stages:

$$\begin{aligned} \text{predict} & \begin{cases} \hat{\mathbf{x}}_{t|t-1} = \mathbf{F}_t \hat{\mathbf{x}}_{t-1|t-1} \\ \mathbf{P}_{t|t-1} = \mathbf{F}_t \mathbf{P}_{t-1|t-1} \mathbf{F}_t^\top + \mathbf{Q}_t \end{cases} \\ \text{update} & \begin{cases} \mathbf{K}_t = \mathbf{P}_{t|t-1} \mathbf{H}_t^\top (\mathbf{H}_t \mathbf{P}_{t|t-1} \mathbf{H}_t^\top + \mathbf{R}_t)^{-1} \\ \hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t (\mathbf{z}_t - \mathbf{H}_t \hat{\mathbf{x}}_{t|t-1}) \\ \mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t \mathbf{H}_t) \mathbf{P}_{t|t-1} \end{cases} \end{aligned} \quad (1)$$

The stage of *predict* is to derive the state estimations on the next time step t . Given a measurement of target states on the next step t , the stage of *update* aims to update the posteriori parameters in KF. Because the measurement comes from the observation model $\mathbf{H}$, it is also called “observation” in many scenarios.

SORT [3] is a multi-object tracker built upon KF. The KF’s state $\mathbf{x}$ in SORT is defined as $\mathbf{x} = [u, v, s, r, \dot{u}, \dot{v}, \dot{s}]^\top$, where (u, v) is the 2D coordinates of the object center in the image. s is the bounding box scale (area) and r is the bounding box aspect ratio. The aspect ratio r is assumed to be constant. The other three variables, $\dot{u}$, $\dot{v}$ and $\dot{s}$ are the corresponding time derivatives. The observation is a bounding box $\mathbf{z} = [u, v, w, h, c]^\top$ with object center position (u, v) , object width w , and height h and the detection confidence c respectively. SORT assumes linear motion as the transition model $\mathbf{F}$ which leads to the state estimation as

$$u_{t+1} = u_t + \dot{u}_t \Delta t, \quad v_{t+1} = v_t + \dot{v}_t \Delta t. \quad (2)$$

To leverage KF (Eq 1) in SORT for visual MOT, the stage of *predict* corresponds to estimating the object position on the next video frame. And the observations used for the update stage usually come from a detection model. The update stage is to update Kalman filter parameters and does not directly edit the tracking outcomes.

When the time difference between two steps is constant during the transition, *e.g.*, the video frame rate is constant,

we can set $\Delta t = 1$. When the video frame rate is high, SORT works well even when the object motion is non-linear globally, (*e.g.* dancing, fencing, wrestling) because the motion of the target object can be well approximated as linear within short time intervals. However, in practice, observations are often absent on some time steps, *e.g.* the target object is occluded in multi-object tracking. In such cases, we cannot update the KF parameters by the update operation as in Eq. 1 anymore. SORT uses the priori estimations directly as posterior. We call this “**dummy update**”, namely

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1}, \mathbf{P}_{t|t} = \mathbf{P}_{t|t-1}. \quad (3)$$

The philosophy behind such a design is to trust estimations when no observations are available to supervise them. We thus call the tracking algorithms following this scheme “estimation-centric”. However, we will see that this estimation-centric mechanism can cause trouble when non-linear motion and occlusion happen together.

3.2. Limitations of SORT

In this section, we identify three main limitations of SORT which are connected. This analysis lays the foundation of our proposed method.

3.2.1 Sensitive to State Noise

Now we show that SORT is sensitive to the noise from KF’s state estimations. To begin with, we assume that the estimated object center position follows $u \sim \mathcal{N}(\mu_u, \sigma_u^2)$ and $v \sim \mathcal{N}(\mu_v, \sigma_v^2)$, where (μ_u, μ_v) is the underlying true position. Then, if we assume that the state noises are independent on different steps, by Eq.2, the object speed between two time steps, $t \rightarrow t + \Delta t$, is

$$\dot{u} = \frac{u_{t+\Delta t} - u_t}{\Delta t}, \quad \dot{v} = \frac{v_{t+\Delta t} - v_t}{\Delta t}, \quad (4)$$

making the noise of estimated speed $\delta_{\dot{u}} \sim \mathcal{N}(0, \frac{2\sigma_u^2}{(\Delta t)^2})$, $\delta_{\dot{v}} \sim \mathcal{N}(0, \frac{2\sigma_v^2}{(\Delta t)^2})$. Therefore, a small Δt will amplify the noise. This suggests that SORT will suffer from the heavy noise of velocity estimation on high-frame-rate videos. The analysis above is simplified from the reality. In practice, velocity won’t be determined by the state on future time steps. For a more strict analysis, please refer to Appendix G.

Moreover, for most multi-object tracking scenarios, the target object displacement is only a few pixels between consecutive frames. For instance, the average displacement is 1.93 pixels and 0.65 pixels along the image width and height for the MOT17 [41] training dataset. In such a case, even if the estimated position has a shift of only a single pixel, it causes a significant variation in the estimated speed. In general, the variance of the speed estimation can be of the same magnitude as the speed itself or even greater. This will not make a massive impact as the shift is only of few pixels

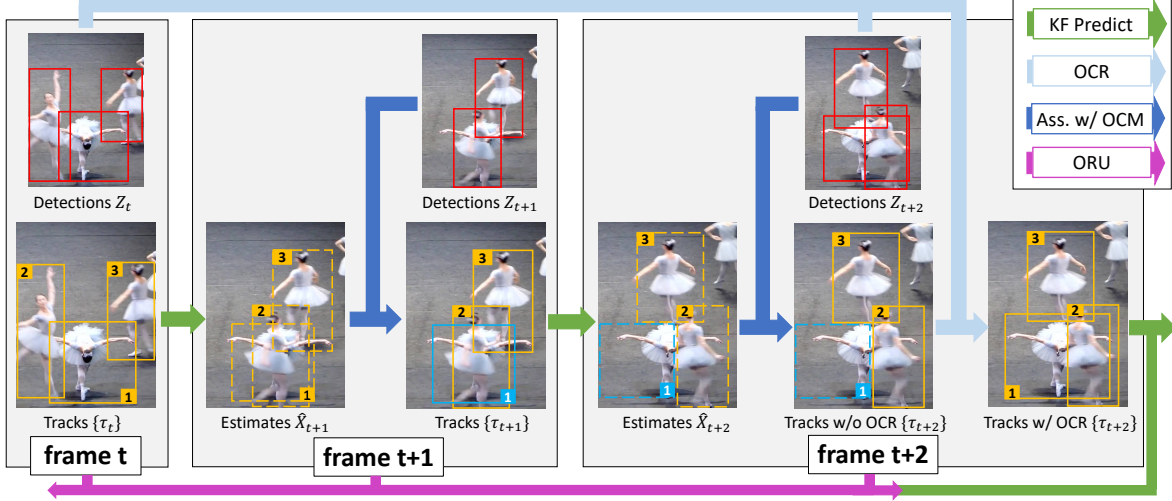


Figure 2. The pipeline of our proposed OC-SORT. The red boxes are detections, orange boxes are active tracks, blue boxes are untracked tracks, and dashed boxes are the estimates from KF. During association, OCM is used to add the velocity consistency cost. The target #1 is lost on the frame t+1 because of occlusion. But on the next frame, it is recovered by referring to its observation of the frame t by OCR. It being re-tracked triggers ORU from t to t+2 for the parameters of its KF.

from the ground truth on the next time step and the observations, whose variance is independent of the time, will be able to fix the noise when updating the posteriori parameters. However, we find that such a high sensitivity to state noise introduces significant problems in practice after being amplified by the error accumulation across multiple time steps when no observation is available for KF *update*.

3.2.2 Temporal Error Magnification

For analysis above in Eq. 4, we assume the noise of the object state is i.i.d on different time steps (this is a simplified version, a more detailed analysis is provided in Appendix G). This is reasonable for object detections but not for the estimations from KF. This is because KF’s estimations always rely on its estimations on previous time steps. The effect is usually minor because KF can use observation in *update* to prevent the posteriori state estimation and covariance, *i.e.* $\hat{x}_{t|t}$ and $\mathbf{P}_{t|t}$, deviating from the true value too far away. However, when no observations are provided to KF, it cannot use observation to update its parameters. Then it has to follow Eq. 3 to prolong the estimated trajectory to the next time step. Consider a track is occluded on the time steps between t and $t+T$ and the noise of speed estimate follows $\delta_{u_t} \sim \mathcal{N}(0, 2\sigma_u^2)$, $\delta_{v_t} \sim \mathcal{N}(0, 2\sigma_v^2)$ for SORT. On the step $t+T$, state estimation would be

$$u_{t+T} = u_t + T\dot{u}_t, \quad v_{t+T} = v_t + T\dot{v}_t, \quad (5)$$

whose noise follows $\delta_{u_{t+T}} \sim \mathcal{N}(0, 2T^2\sigma_u^2)$ and $\delta_{v_{t+T}} \sim \mathcal{N}(0, 2T^2\sigma_v^2)$. So without the observations, the estimation from the linear motion assumption of KF results in a fast error accumulation with respect to time. Given σ_v and σ_u is of the same magnitude as object displacement between consecutive frames, the noise of final object position

(u_{t+T}, v_{t+T}) is of the same magnitude as the object size. For instance, the size of pedestrians close to the camera on MOT17 is around 50×300 pixels. So even assuming the variance of position estimation is only 1 pixel, 10-frame occlusion can accumulate a shift in final position estimation as large as the object size. Such error magnification leads to a major accumulation of errors when the scenes are crowded.

3.2.3 Estimation-Centric

The aforementioned limitations come from a fundamental property of SORT that it follows KF to be estimation-centric. It allows *update* without the existence of observations and purely trusts the estimations. A key difference between state estimations and observations is that we can assume that the observations by an object detector in each frame are affected by i.i.d. noise $\delta_z \sim \mathcal{N}(0, \sigma'^2)$ while the noise in state estimations can be accumulated along the hidden Markov process. Moreover, modern object detectors use powerful object visual features [48, 51]. It makes that, even on a single frame, it is usually safe to assume $\sigma' < \sigma_u$ and $\sigma' < \sigma_v$ because the object localization is more accurate by detection than from the state estimations through linear motion assumption. Combined with the previously mentioned two limitations, being estimation-centric makes SORT suffer from heavy noise when there is occlusion and the object motion is not perfectly linear.

4. Observation-Centric SORT

In this section, we introduce the proposed *Observation-Centric SORT (OC-SORT)*. To address the limitations of SORT discussed above, we use the momentum of the object moving into the association stage and develop a

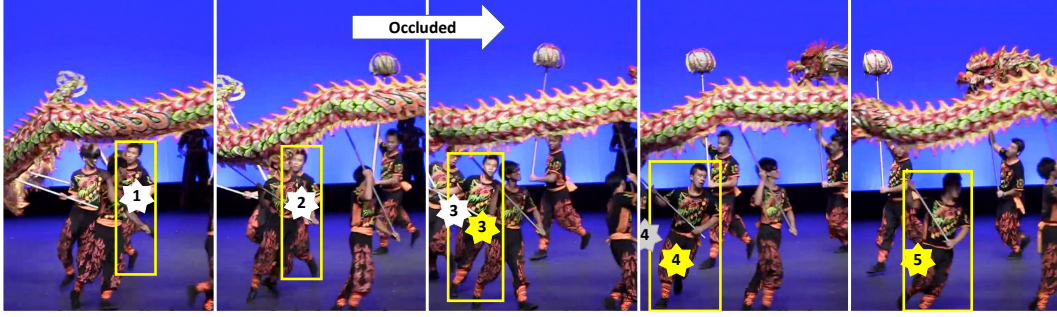


Figure 3. Example of how *Observation-centric Re-Update (ORU)* reduces the error accumulation when a track is broken. The target is occluded between the second and the third time step and the tracker finds it back at the third step. Yellow boxes are the state observations by the detector. White stars are the estimated centers without ORU. Yellow stars are the estimated centers fixed by ORU. The gray star on the fourth step is the estimated center without ORU and fails to match observations.

pipeline with less noise and more robustness over occlusion and non-linear motion. The key is to design the tracker as **observation-centric** instead of **estimation-centric**. If a track is recovered from being untracked, we use an *Observation-centric Re-Update (ORU)* strategy to counter the accumulated error during the untracked period. OC-SORT also adds an *Observation-Centric Momentum (OCM)* term in the association cost. Please refer to Algorithm 1 in Appendix for the pseudo-code of OC-SORT. The pipeline is shown in Fig. 2.

4.1. Observation-centric Re-Update (ORU)

In practice, even if an object can be associated again by SORT after a period of being untracked, it is probably lost again because its KF parameters have already deviated far away from the correct due to the temporal error magnification. To alleviate this problem, we propose *Observation-centric Re-Update (ORU)* to reduce the accumulated error. Once a track is associated with an observation again after a period of being untracked (“re-activation”), we backcheck the period of its being lost and re-update the parameters of KF. The re-update is based on “observations” from a virtual trajectory. The virtual trajectory is generated referring to the observations on the steps starting and ending the untracked period. For example, by denoting the last-seen observation before being untracked as $\mathbf{z}_{t_1}$ and the observation triggering the re-association as $\mathbf{z}_{t_2}$, the virtual trajectory is denoted as

$$\tilde{\mathbf{z}}_t = \text{Traj}_{\text{virtual}}(\mathbf{z}_{t_1}, \mathbf{z}_{t_2}, t), t_1 < t < t_2. \quad (6)$$

Then, along the trajectory of $\tilde{\mathbf{z}}_t (t_1 < t < t_2)$, we run the loop of *predict* and *re-update*. The *re-update* operation is

$$\text{re-update} \begin{cases} \mathbf{K}_t = \mathbf{P}_{t|t-1} \mathbf{H}_t^\top (\mathbf{H}_t \mathbf{P}_{t|t-1} \mathbf{H}_t^\top + \mathbf{R}_t)^{-1} \\ \hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t (\tilde{\mathbf{z}}_t - \mathbf{H}_t \hat{\mathbf{x}}_{t|t-1}) \\ \mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t \mathbf{H}_t) \mathbf{P}_{t|t-1} \end{cases} \quad (7)$$

As the observations on the virtual trajectory match the motion pattern anchored by the last-seen and the latest asso-

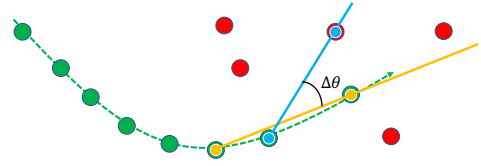


Figure 4. Calculation of motion direction difference in OCM. The green line indicates an existing track and the dots are the observations on it. The red dots are the new observations to be associated. The blue link and the yellow link form the directions of θ^{track} and $\theta^{\text{intention}}$ respectively. The included angle is the difference of direction $\Delta\theta$.

ciation real observations, the update will not suffer from the error accumulated through the dummy update anymore. We call the proposed process *Observation-centric Re-Update*. It serves as an independent stage outside the *predict-update* loop and is triggered only a track is re-activated from a period of having no observations.

4.2. Observation-Centric Momentum (OCM)

In a reasonably short time interval, we can approximate the motion as linear. And the linear motion assumption also asks for consistent motion direction. But the noise prevents us from leveraging the consistency of direction. To be precise, to determine the motion direction, we need the object state on two steps with a time difference Δt . If Δt is small, the velocity noise would be significant because of the estimation’s sensitivity to state noise. If Δt is big, the noise of direction estimation can also be significant because of the temporal error magnification and the failure of linear motion assumption. As state observations have no problem of temporal error magnification that state estimations suffer from, we propose to use observations instead of estimations to reduce the noise of motion direction calculation and introduce the term of its consistency to help the association.

With the new term, given N existing tracks and M detections on the new-coming time step, the association cost

matrix is formulated as

$$C(\hat{\mathbf{X}}, \mathbf{Z}) = C_{\text{IoU}}(\hat{\mathbf{X}}, \mathbf{Z}) + \lambda C_v(\mathbf{Z}, \mathbf{Z}), \quad (8)$$

where $\hat{\mathbf{X}} \in \mathbb{R}^{N \times 7}$ is the set of object state estimations and $\mathbf{Z} \in \mathbb{R}^{M \times 5}$ is the set of observations on the new time step. λ is a weighting factor. $\mathbf{Z}$ contains the trajectory of observations of all existing tracks. $C_{\text{IoU}}(\cdot, \cdot)$ calculates the negative pairwise IoU (Intersection over Union) and $C_v(\cdot, \cdot)$ calculates the consistency between the directions of i) linking two observations on an existing track (θ^{track}) and ii) linking a track’s historical observation and a new observation ($\theta^{\text{intention}}$). C_v contains all pairs of $\Delta\theta = |\theta^{\text{track}} - \theta^{\text{intention}}|$. In our implementation, we calculate the motion direction in radians, namely $\theta = \arctan(\frac{v_1 - v_2}{u_1 - u_2})$ where (u_1, v_1) and (u_2, v_2) are the observations on two different time steps. The calculation of this is also illustrated in Figure 4.

Following the assumptions of noise distribution mentioned before, we can derive a closed-form probability density function of the distribution of the noise in the direction estimation. The derivation is explained in detail in Appendix A. By analyzing the property of this distribution, we reach a conclusion that, under the linear-motion model, the scale of the noise of direction estimation is negatively correlated to the time difference between the two observation points, *i.e.* Δt . This suggests increasing Δt to achieve a low-noisy estimation of θ . However, the assumption of linear motion typically holds only when Δt is small enough. Therefore, the choice of Δt requires a trade-off.

Besides ORU and OCM, we also find it empirically helpful to check a track’s last presence to recover it from being lost. We thus apply a heuristic *Observation-Centric Recovery (OCR)* technique. OCR will start a second attempt of associating between the last observation of unmatched tracks to the unmatched observations after the usual association stage. It can handle the case of an object stopping or being occluded for a short time interval.

5. Experiments

5.1. Experimental Setup

Datasets. We evaluate our method on multiple multi-object tracking datasets including MOT17 [41], MOT20 [14], KITTI [20], DanceTrack [54] and CroHD [56]. MOT17 [41] and MOT20 [14] are for pedestrian tracking, where targets mostly move linearly, while scenes in MOT20 are more crowded. KITTI [20] is for pedestrian and car tracking with a relatively low frame rate of 10FPS. CroHD is a dataset for head tracking in the crowd and the results on it are included in the appendix. DanceTrack [54] is a recently proposed dataset for human tracking. For the data in DanceTrack, object localization is easy, but the object motion is highly non-linear. Furthermore, the objects have a close appearance, severe occlusion, and frequent

crossovers. Considering our goal is to improve tracking robustness under occlusion and non-linear object motion, we would emphasize the comparison on DanceTrack.

Implementations. For a fair comparison, we directly apply the object detections from existing baselines. For MOT17, MOT20, and DanceTrack, we use the publicly available YOLOX [19] detector weights by ByteTrack [70]. For KITTI [20], we use the detections from PermaTrack [57] publicly available in the official release¹. For ORU, we generate the virtual trajectory during occlusion with the constant-velocity assumption. Therefore, Eq. 6 is adopted as $\tilde{\mathbf{z}}_t = \mathbf{z}_{t_1} + \frac{t-t_1}{t_2-t_1}(\mathbf{z}_{t_2} - \mathbf{z}_{t_1})$, $t_1 < t < t_2$. For OCM, the velocity direction is calculated using the observations three time steps apart, *i.e.* $\Delta t = 3$. The direction difference is measured by the absolute difference of angles in radians. We set $\lambda = 0.2$ in Eq. 8. Following the common practice of SORT, we set the detection confidence threshold at 0.4 for MOT20 and 0.6 for other datasets. The IoU threshold during association is 0.3.

Metrics. We adopt HOTA [37] as the main metric as it maintains a better balance between the accuracy of object detection and association [37]. We also emphasize AssA to evaluate the association performance. IDF1 is also used for association performance evaluation. Other metrics we report, such as MOTA, are highly related to detection performance. It is fair to use these metrics only when all methods use the same detections for tracking, which is referred to as “public tracking” as reported in Appendix C.

5.2. Benchmark Results

Here we report the benchmark results on multiple datasets. We put all methods that use the shared detection results in a block at the bottom of each table.

MOT17 and MOT20. We report OC-SORT’s performance on MOT17 and MOT20 in Table 1 and Table 2 using private detections. To make a fair comparison, we use the same detection as ByteTrack [70]. OC-SORT achieves performance comparable to other state-of-the-art methods. Our gains are especially significant in MOT20 under severe pedestrian occlusion, setting a state-of-the-art HOTA of 62.1. As our method is designed to be simple for better generalization, we do not use adaptive detection thresholds as in ByteTrack. Also, ByteTrack uses more detections of low-confidence to achieve higher MOTA scores but we keep the detection confidence threshold the same as on other datasets, which is the common practice in the community. We inherit the linear interpolation on the two datasets as baseline methods for a fair comparison. To more clearly discard the variance from the detector, we also perform public tracking on MOT17 and MOT20, which is reported in Table 12 and Table 13 in Appendix C. OC-SORT still outperforms the existing state-of-the-art in public tracking settings.

¹<https://github.com/TRI-ML/permatrack/>

Table 1. Results on MOT17-test with the private detections. ByteTrack and OC-SORT share detections.

Tracker	HOTA $\uparrow$	MOTA $\uparrow$	IDF1 $\uparrow$	FP(10^4) $\downarrow$	FN(10^4) $\downarrow$	IDs $\downarrow$	Frag $\downarrow$	AssA $\uparrow$	AssR $\uparrow$
FairMOT [71]	59.3	73.7	72.3	2.75	11.7	3,303	8,073	58.0	63.6
TransCt [67]	54.5	73.2	62.2	2.31	12.4	4,614	9,519	49.7	54.2
TransTrk [55]	54.1	75.2	63.5	5.02	8.64	3,603	4,872	47.9	57.1
GRTU [60]	62.0	74.9	75.0	3.20	10.8	1,812	1,824	62.1	65.8
QDTrack [42]	53.9	68.7	66.3	2.66	14.7	3,378	8,091	52.7	57.2
MOTR [69]	57.2	71.9	68.4	2.11	13.6	2,115	3,897	55.8	59.2
PermaTr [57]	55.5	73.8	68.9	2.90	11.5	3,699	6,132	53.1	59.8
TransMOT [12]	61.7	76.7	75.1	3.62	9.32	2,346	7,719	59.9	66.5
GTR [75]	59.1	75.3	71.5	2.68	11.0	2,859	-	61.6	-
DST-Tracker [8]	60.1	75.2	72.3	2.42	11.0	2,729	-	62.1	-
MeMOT [5]	56.9	72.5	69.0	2.72	11.5	2,724	-	55.2	-
UniCorn [68]	61.7	77.2	75.5	5.01	7.33	5,379	-	-	-
ByteTrack [70]	63.1	80.3	77.3	2.55	8.37	2,196	2,277	62.0	68.2
OC-SORT	63.2	78.0	77.5	1.51	10.8	1,950	2,040	63.2	67.5

Table 2. Results on MOT20-test with private detections. ByteTrack and OC-SORT share detections.

Tracker	HOTA $\uparrow$	MOTA $\uparrow$	IDF1 $\uparrow$	FP(10^4) $\downarrow$	FN(10^4) $\downarrow$	IDs $\downarrow$	Frag $\downarrow$	AssA $\uparrow$	AssR $\uparrow$
FairMOT [71]	54.6	61.8	67.3	10.3	8.89	5,243	7,874	54.7	60.7
TransCt [67]	43.5	58.5	49.6	6.42	14.6	4,695	9,581	37.0	45.1
Semi-TCL [35]	55.3	65.2	70.1	6.12	11.5	4,139	8,508	56.3	60.9
CSTrack [36]	54.0	66.6	68.6	2.54	14.4	3,196	7,632	54.0	57.6
GSDT [61]	53.6	67.1	67.5	3.19	13.5	3,131	9,875	52.7	58.5
TransMOT [12]	61.9	77.5	75.2	3.42	8.08	1,615	2,421	60.1	66.3
MeMOT [5]	54.1	63.7	66.1	4.79	13.8	1,938	-	55.0	-
ByteTrack [70]	61.3	77.8	75.2	2.62	8.76	1,223	1,460	59.6	66.2
OC-SORT	62.1	75.5	75.9	1.80	10.8	913	1,198	62.0	67.5

Table 3. Results on DanceTrack test set. Methods in the blue block share the same detections.

Tracker	HOTA $\uparrow$	DetA $\uparrow$	AssA $\uparrow$	MOTA $\uparrow$	IDF1 $\uparrow$
CenterTrack [73]	41.8	78.1	22.6	86.8	35.7
FairMOT [71]	39.7	66.7	23.8	82.2	40.8
QDTrack [42]	45.7	72.1	29.2	83.0	44.8
TransTrk[55]	45.5	75.9	27.5	88.4	45.2
TraDes [64]	43.3	74.5	25.4	86.2	41.2
MOTR [69]	54.2	73.5	40.2	79.7	51.5
GTR [75]	48.0	72.5	31.9	84.7	50.3
DST-Tracker [8]	51.9	72.3	34.6	84.9	51.0
SORT [3]	47.9	72.0	31.2	91.8	50.8
DeepSORT [63]	45.6	71.0	29.7	87.8	47.9
ByteTrack [70]	47.3	71.6	31.4	89.5	52.5
OC-SORT	54.6	80.4	40.2	89.6	54.6
OC-SORT + Linear Interp	55.1	80.4	40.4	92.2	54.9

DanceTrack. To evaluate OC-SORT under challenging non-linear object motion, we report results on the DanceTrack in Table 3. OC-SORT sets a new state-of-the-art, outperforming the baselines by a great margin under non-linear object motions. We compare the tracking results of SORT and OC-SORT under extreme non-linear situations in Fig.1 and more samples are available in Fig. 8 in Appendix E. We also visualize the output trajectories by OC-SORT and SORT on randomly selected DanceTrack video clips in Fig. 9 in Appendix E. For multi-object tracking in occlusion and non-linear motion, the results on DanceTrack are strong evidence of the effectiveness of OC-SORT.

KITTI. In Table 4 we report the results on the KITTI dataset. For a fair comparison, we adopt the detector weights by PermaTr [57] and report its performance in the table as well. Then, we run OC-SORT given the shared detections. As initializing SORT’s track requires continuous

tracking across several frames (“minimum hits”), we observe that the results not recorded during the track initialization make a significant difference. To address this problem, we perform offline head padding (HP) post-processing by writing these entries back after finishing the online tracking stage. The results of the car category on KITTI show an essential shortcoming of the default implementation version of OC-SORT that it chooses the IoU matching for the association. When the object velocity is high or the frame rate is low, the IoU of object bounding boxes between consecutive frames can be very low or even zero. This issue does not come from the intrinsic design of OC-SORT and is widely observed when using IoU as the association cue. Adding other cues [49, 72, 73] and appearance similarity [38, 63] have been demonstrated [63] efficient to solve this. In contrast to the relatively inferior car tracking performance, OC-SORT improves pedestrian tracking performance to a new state-of-the-art. Using the same detections, OC-SORT achieves a large performance gap over PermaTr with 10x faster speed.

The results on multiple benchmarks have demonstrated the effectiveness and efficiency of OC-SORT. We note that we use a shared parameter stack across datasets. Carefully tuning the parameters can probably further boost the performance. For example, the adaptive detection threshold is proven useful in previous work [70]. Besides the association accuracy, we also care about the inference speed. Given off-the-shelf detections, OC-SORT runs at 793 FPS on an Intel i9-9980XE CPU @ 3.00GHz. Therefore, OC-SORT can still run in an online and real-time fashion.

Table 4. Results on KITTI-test. Our method uses the same detections as PermaTr [57]

Tracker	Car					Pedestrian				
	HOTA↑	MOTA↑	AssA↑	IDs↓	Frag↓	HOTA↑	MOTA↑	AssA↑	IDs↓	Frag↓
IMMDP [65]	68.66	82.75	69.76	211	181	-	-	-	-	-
SMAT [21]	71.88	83.64	72.13	198	294	-	-	-	-	-
TrackMPNN [45]	72.30	87.33	70.63	481	237	39.40	52.10	35.45	626	669
MPNTrack [4]	-	-	-	-	-	45.26	46.23	47.28	397	1,078
CenterTr [73]	73.02	88.83	71.18	254	227	40.35	53.84	36.93	425	618
LGM [59]	73.14	87.60	72.31	448	164	-	-	-	-	-
TuSimple [11]	71.55	86.31	71.11	292	218	45.88	57.61	47.62	246	651
PermaTr [57]	77.42	90.85	77.66	275	271	47.43	65.05	43.66	483	703
OC-SORT	74.64	87.81	74.52	257	318	52.95	62.00	57.81	181	598
OC-SORT + HP	76.54	90.28	76.39	250	280	54.69	65.14	59.08	184	609

Table 5. Ablation on MOT17-val and DanceTrack-val.

			MOT17-val			DanceTrack-val		
ORU	OCM	OCR	HOTA↑	AssA↑	IDF1↑	HOTA↑	AssA↑	IDF1↑
			64.9	66.8	76.9	47.8	31.0	48.3
✓			66.3	68.0	77.2	48.5	32.2	49.8
✓	✓		66.4	69.0	77.8	52.1	35.0	50.6
✓	✓	✓	66.5	68.9	77.7	52.1	35.3	51.6

Table 6. Ablation on the trajectory hypothesis in ORU.

			MOT17-val			DanceTrack-val		
			HOTA↑	AssA↑	IDF1↑	HOTA↑	AssA↑	IDF1↑
Const. Speed			66.5	68.9	77.7	52.1	35.3	51.6
GPR			63.1	65.2	75.7	49.5	33.7	49.6
Linear Regression			64.3	66.5	76.0	49.3	33.4	49.2
Const. Acceleration			66.2	67.9	77.4	51.3	34.8	50.9

Table 7. Influence from the value of Δt in OCM.

			MOT17-val			DanceTrack-val		
			HOTA↑	AssA↑	IDF1↑	HOTA↑	AssA↑	IDF1↑
$\Delta t = 1$			66.1	67.5	76.9	51.3	34.3	51.3
$\Delta t = 2$			66.3	68.0	77.3	52.2	35.4	51.4
$\Delta t = 3$			66.5	68.9	77.7	52.1	35.3	51.6
$\Delta t = 6$			66.0	67.5	76.9	52.1	35.4	51.8

5.3. Ablation Study

Component Ablation. We ablate the contribution of proposed modules on the validation sets of MOT17 and DanceTrack in Table 5. The splitting of the MOT17 validation set follows a popular convention [73]. The results demonstrate the efficiency of the proposed modules in OC-SORT. The results show that the performance gain from ORU is significant on both datasets but OCM only shows good help on DanceTrack dataset where object motion is more complicated and the occlusion is heavy. It suggests the effectiveness of our proposed method to improve tracking robustness in occlusion and non-linear motion.

Virtual Trajectory in ORU. For simplicity, we follow the naive hypothesis of constant speed to generate a virtual trajectory in ORU. There are other alternatives like constant acceleration, regression-based fitting such as Linear Regression (LR) or Gaussian Process Regression (GPR), and Near Constant Acceleration Model (NCAM) [27]. The results of comparing these choices are shown in Table 6. For GPR,

we use the RBF kernel [10] $k(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{50}\right)$.

We provide more studies on the kernel configuration in Appendix B. The results show that local hypotheses such as Constant Speed/Acceleration perform much better than global hypotheses such as LR and GPR. This is probably because, as virtual trajectory generation happens in an on-line fashion, it is hard to get a reliable fit using only limited data points on historical time steps.

Δt in OCM. There is a trade-off when choosing the time difference Δt in OCM (Section 4). A large Δt decreases the noise of velocity estimation. but is also likely to discourage approximating object motion as linear. Therefore, we study the influence of varying Δt in Table 7. Our results agree with our analysis that increasing Δt from $\Delta t = 1$ can boost the association performance. But increasing Δt higher than the bottleneck instead hurts the performance because of the difficulty of maintaining the approximation of linear motion.

6. Conclusion

We analyze the popular motion-based tracker SORT and recognize its intrinsic limitations from using Kalman filter. These limitations significantly hurt tracking accuracy when the tracker fails to gain observations for supervision - likely caused by unreliable detectors, occlusion, or fast and non-linear target object motion. To address these issues, we propose *Observation-Centric SORT (OC-SORT)*. OC-SORT is more robust to occlusion and non-linear object motion while keeping simple, online, and real-time. In our experiments on diverse datasets, OC-SORT significantly outperforms the state-of-the-art. The gain is especially significant for multi-object tracking under occlusion and non-linear object motion.

Acknowledgement. We thank David Held, Deva Ramanan, and Siddharth Ancha for the discussion about the theoretical modeling of OC-SORT. We thank Yuda Song for the help with the analysis of OCM error distribution. We also would like to thank Zhengyi Luo, Yuda Song, and Erica Weng for the detailed feedback on the paper writing. We also thank Yifu Zhang for sharing his experience with ByteTrack. This project was sponsored in part by NSF NRI award 2024173.

References

- [1] Yaakov Bar-Shalom, Fred Daum, and Jim Huang. The probabilistic data association filter. *IEEE Control Systems Magazine*, 29(6):82–100, 2009.
- [2] Sumit Basu, Irfan Essa, and Alex Pentland. Motion regularization for model-based head tracking. In *Proceedings of 13th International Conference on Pattern Recognition*, volume 3, pages 611–616. IEEE, 1996.
- [3] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Uprocroft. Simple online and realtime tracking. In *2016 IEEE international conference on image processing (ICIP)*, pages 3464–3468. IEEE, 2016.
- [4] Guillem Brasó and Laura Leal-Taixé. Learning a neural solver for multiple object tracking. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6247–6257, 2020.
- [5] Jiarui Cai, Mingze Xu, Wei Li, Yuanjun Xiong, Wei Xia, Zhuowen Tu, and Stefano Soatto. Memot: multi-object tracking with memory. In *CVPR*, 2022.
- [6] Jinkun Cao, Jiangmiao Pang, Xinshuo Weng, Rawal Khirodkar, and Kris M Kitani. Object tracking by hierarchical part-whole attention.
- [7] Jinkun Cao, Hongyang Tang, Hao-Shu Fang, Xiaoyong Shen, Cewu Lu, and Yu-Wing Tai. Cross-domain adaptation for animal pose estimation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9498–9507, 2019.
- [8] Jinkun Cao, Hao Wu, and Kris Kitani. Track targets by dense spatio-temporal position encoding. *arXiv preprint arXiv:2210.09455*, 2022.
- [9] Ming-Fang Chang, John Lambert, Patsorn Sangkloy, Jagjeet Singh, Slawomir Bak, Andrew Hartnett, De Wang, Peter Carr, Simon Lucey, Deva Ramanan, et al. Argoverse: 3d tracking and forecasting with rich maps. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8748–8757, 2019.
- [10] Yin-Wen Chang, Cho-Jui Hsieh, Kai-Wei Chang, Michael Ringgaard, and Chih-Jen Lin. Training and testing low-degree polynomial data mappings via linear svm. *Journal of Machine Learning Research*, 11(4), 2010.
- [11] Wongun Choi. Near-online multi-target tracking with aggregated local flow descriptor. In *Proceedings of the IEEE international conference on computer vision*, pages 3029–3037, 2015.
- [12] Peng Chu, Jiang Wang, Quanzeng You, Haibin Ling, and Zicheng Liu. Transmot: Spatial-temporal graph transformer for multiple object tracking. *arXiv preprint arXiv:2104.00194*, 2021.
- [13] Peng Dai, Renliang Weng, Wongun Choi, Changshui Zhang, Zhangping He, and Wei Ding. Learning a proposal classifier for multiple object tracking. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2443–2452, 2021.
- [14] Patrick Dendorfer, Hamid Rezatofighi, Anton Milan, Javen Shi, Daniel Cremers, Ian Reid, Stefan Roth, Konrad Schindler, and Laura Leal-Taixé. Mot20: A benchmark for multi object tracking in crowded scenes. *arXiv preprint arXiv:2003.09003*, 2020.
- [15] David Duvenaud. *Automatic model construction with Gaussian processes*. PhD thesis, University of Cambridge, 2014.
- [16] Hao-Shu Fang, Jinkun Cao, Yu-Wing Tai, and Cewu Lu. Pairwise body-part attention for recognizing human-object interactions. In *Proceedings of the European conference on computer vision (ECCV)*, pages 51–67, 2018.
- [17] Juergen Gall, Angela Yao, Nima Razavi, Luc Van Gool, and Victor Lempitsky. Hough forests for object detection, tracking, and action recognition. *IEEE transactions on pattern analysis and machine intelligence*, 33(11):2188–2202, 2011.
- [18] Damien Garreau, Wittawat Jitkrittum, and Motonobu Kanagawa. Large sample analysis of the median heuristic. *arXiv preprint arXiv:1707.07269*, 2017.
- [19] Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. YOLOX: Exceeding yolo series in 2021. *arXiv preprint arXiv:2107.08430*, 2021.
- [20] Andreas Geiger, Philip Lenz, Christoph Stiller, and Raquel Urtasun. Vision meets robotics: The kitti dataset. *The International Journal of Robotics Research*, 32(11):1231–1237, 2013.
- [21] Nicolas Franco Gonzalez, Andres Ospina, and Philippe Calvez. Smat: Smart multiple affinity metrics for multiple object tracking. In *International Conference on Image Analysis and Recognition*, pages 48–62. Springer, 2020.
- [22] Fredrik Gustafsson, Fredrik Gunnarsson, Niclas Bergman, Urban Forssell, Jonas Jansson, Rickard Karlsson, and P-J Nordlund. Particle filters for positioning, navigation, and tracking. *IEEE Transactions on signal processing*, 50(2):425–437, 2002.
- [23] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [24] David V Hinkley. On the ratio of two correlated normal random variables. *Biometrika*, 56(3):635–639, 1969.
- [25] Andrea Hornakova, Roberto Henschel, Bodo Rosenhahn, and Paul Swoboda. Lifted disjoint paths with application in multiple object tracking. In *International Conference on Machine Learning*, pages 4364–4375. PMLR, 2020.
- [26] Andrea Hornakova, Timo Kaiser, Paul Swoboda, Michal Rolínek, Bodo Rosenhahn, and Roberto Henschel. Making higher order mot scalable: An efficient approximate solver for lifted disjoint paths. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6330–6340, 2021.
- [27] Andrew H Jazwinski. *Stochastic processes and filtering theory*. Courier Corporation, 2007.
- [28] Simon J Julier and Jeffrey K Uhlmann. New extension of the kalman filter to nonlinear systems. In *Signal processing, sensor fusion, and target recognition VI*, volume 3068, pages 182–193. International Society for Optics and Photonics, 1997.
- [29] Simon J Julier and Jeffrey K Uhlmann. Unscented filtering and nonlinear estimation. *Proceedings of the IEEE*, 92(3):401–422, 2004.
- [30] Rudolf Emil Kalman et al. Contributions to the theory of optimal control. *Bol. soc. mat. mexicana*, 5(2):102–119, 1960.
- [31] Kris M Kitani, Brian D Ziebart, James Andrew Bagnell, and Martial Hebert. Activity forecasting. In *European conference on computer vision*, pages 201–214. Springer, 2012.

- [32] Jonathan Ko and Dieter Fox. Gp-bayesfilters: Bayesian filtering using gaussian process prediction and observation models. *Autonomous Robots*, 27(1):75–90, 2009.
- [33] Parth Kothari, Sven Kreiss, and Alexandre Alahi. Human trajectory forecasting in crowds: A deep learning perspective. *IEEE Transactions on Intelligent Transportation Systems*, 2021.
- [34] Erich L Lehmann and George Casella. *Theory of point estimation*. Springer Science & Business Media, 2006.
- [35] Wei Li, Yuanjun Xiong, Shuo Yang, Mingze Xu, Yongxin Wang, and Wei Xia. Semi-tcl: Semi-supervised track contrastive representation learning. *arXiv preprint arXiv:2107.02396*, 2021.
- [36] Chao Liang, Zhipeng Zhang, Yi Lu, Xue Zhou, Bing Li, Xiyong Ye, and Jianxiao Zou. Rethinking the competition between detection and reid in multi-object tracking. *arXiv preprint arXiv:2010.12138*, 2020.
- [37] Jonathon Luiten, Aljosa Osep, Patrick Dendorfer, Philip Torr, Andreas Geiger, Laura Leal-Taixé, and Bastian Leibe. Hota: A higher order metric for evaluating multi-object tracking. *International journal of computer vision*, 129(2):548–578, 2021.
- [38] Gerard Maggolino, Adnan Ahmad, Jinkun Cao, and Kris Kitani. Deep oc-sort: Multi-pedestrian tracking by adaptive re-identification. *arXiv preprint arXiv:2302.11813*, 2023.
- [39] Tim Meinhardt, Alexander Kirillov, Laura Leal-Taixé, and Christoph Feichtenhofer. Trackformer: Multi-object tracking with transformers. *arXiv preprint arXiv:2101.02702*, 2021.
- [40] Stan Melax, Leonid Keselman, and Sterling Orsten. Dynamics based 3d skeletal hand tracking. In *Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*, pages 184–184, 2013.
- [41] Anton Milan, Laura Leal-Taixé, Ian Reid, Stefan Roth, and Konrad Schindler. Mot16: A benchmark for multi-object tracking. *arXiv preprint arXiv:1603.00831*, 2016.
- [42] Jiangmiao Pang, Linlu Qiu, Xia Li, Haofeng Chen, Qi Li, Trevor Darrell, and Fisher Yu. Quasi-dense similarity learning for multiple object tracking. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 164–173, 2021.
- [43] Dezhi Peng, Zikai Sun, Zirong Chen, Zirui Cai, Lele Xie, and Lianwen Jin. Detecting heads using feature refine net and cascaded multi-scale architecture. *arXiv preprint arXiv:1803.09256*, 2018.
- [44] Lawrence Rabiner and Biinghwang Juang. An introduction to hidden markov models. *ieee assp magazine*, 3(1):4–16, 1986.
- [45] Akshay Rangesh, Pranav Maheshwari, Mez Gebre, Siddhesh Mhatre, Vahid Ramezani, and Mohan M Trivedi. Trackmpnn: A message passing graph neural architecture for multi-object tracking. *arXiv preprint arXiv:2101.04206*, 2021.
- [46] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 779–788, 2016.
- [47] Steven Reece and Stephen Roberts. An introduction to gaussian processes for the kalman filter expert. In *2010 13th International Conference on Information Fusion*, pages 1–9. IEEE, 2010.
- [48] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in neural information processing systems*, 28, 2015.
- [49] Hamid Rezaatoughi, Nathan Tsoi, JunYoung Gwak, Amir Sadeghian, Ian Reid, and Silvio Savarese. Generalized intersection over union: A metric and a loss for bounding box regression. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 658–666, 2019.
- [50] Toby Sharp, Cem Keskin, Duncan Robertson, Jonathan Taylor, Jamie Shotton, David Kim, Christoph Rhemann, Ido Leichter, Alon Vinnikov, Yichen Wei, et al. Accurate, robust, and flexible real-time hand tracking. In *Proceedings of the 33rd annual ACM conference on human factors in computing systems*, pages 3633–3642, 2015.
- [51] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- [52] Gerald L Smith, Stanley F Schmidt, and Leonard A McGee. *Application of statistical filter theory to the optimal estimation of position and velocity on board a circumlunar vehicle*. National Aeronautics and Space Administration, 1962.
- [53] Daniel Stadler and Jurgen Beyerer. Improving multiple pedestrian tracking by track management and occlusion handling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10958–10967, 2021.
- [54] Peize Sun, Jinkun Cao, Yi Jiang, Zehuan Yuan, Song Bai, Kris Kitani, and Ping Luo. Dancetrack: Multi-object tracking in uniform appearance and diverse motion. *arXiv preprint arXiv:2111.14690*, 2021.
- [55] Peize Sun, Jinkun Cao, Yi Jiang, Rufeng Zhang, Enze Xie, Zehuan Yuan, Changhu Wang, and Ping Luo. Transtrack: Multiple object tracking with transformer. *arXiv preprint arXiv:2012.15460*, 2020.
- [56] Ramana Sundararaman, Cedric De Almeida Braga, Eric Marchand, and Julien Pettre. Tracking pedestrian heads in dense crowd. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3865–3875, 2021.
- [57] Pavel Tokmakov, Jie Li, Wolfram Burgard, and Adrien Gaidon. Learning to track with object permanence. In *ICCV*, pages 10860–10869, 2021.
- [58] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [59] Gaoang Wang, Renshu Gu, Zuozhu Liu, Weijie Hu, Mingli Song, and Jenq-Neng Hwang. Track without appearance: Learn box and tracklet embedding with local and global motion patterns for vehicle tracking. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9876–9886, 2021.
- [60] Shuai Wang, Hao Sheng, Yang Zhang, Yubin Wu, and Zhang Xiong. A general recurrent tracking framework without real data. In *Proceedings of the IEEE/CVF International Confer-*

- ence on *Computer Vision*, pages 13219–13228, 2021.
- [61] Yongxin Wang, Kris Kitani, and Xinshuo Weng. Joint object detection and multi-object tracking with graph neural networks. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 13708–13715. IEEE, 2021.
 - [62] Christopher Williams and Carl Rasmussen. Gaussian processes for regression. *Advances in neural information processing systems*, 8, 1995.
 - [63] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. Simple online and realtime tracking with a deep association metric. In *2017 IEEE international conference on image processing (ICIP)*, pages 3645–3649. IEEE, 2017.
 - [64] Jialian Wu, Jiale Cao, Liangchen Song, Yu Wang, Ming Yang, and Junsong Yuan. Track to detect and segment: An online multi-object tracker. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12352–12361, 2021.
 - [65] Yu Xiang, Alexandre Alahi, and Silvio Savarese. Learning to track: Online multi-object tracking by decision making. In *Proceedings of the IEEE international conference on computer vision*, pages 4705–4713, 2015.
 - [66] Yuliang Xiu, Jiefeng Li, Haoyu Wang, Yinghong Fang, and Cewu Lu. Pose flow: Efficient online pose tracking. *arXiv preprint arXiv:1802.00977*, 2018.
 - [67] Yihong Xu, Yutong Ban, Guillaume Delorme, Chuang Gan, Daniela Rus, and Xavier Alameda-Pineda. Transcenter: Transformers with dense queries for multiple-object tracking. *arXiv preprint arXiv:2103.15145*, 2021.
 - [68] Bin Yan, Yi Jiang, Peize Sun, Dong Wang, Zehuan Yuan, Ping Luo, and Huchuan Lu. Towards grand unification of object tracking. In *ECCV*. Springer, 2022.
 - [69] Fangao Zeng and et al. Motr: End-to-end multiple-object tracking with transformer. In *ECCV*. Springer, 2022.
 - [70] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Byte-track: Multi-object tracking by associating every detection box. *arXiv preprint arXiv:2110.06864*, 2021.
 - [71] Yifu Zhang, Chunyu Wang, Xinggang Wang, Wenjun Zeng, and Wenyu Liu. Fairmot: On the fairness of detection and re-identification in multiple object tracking. *International Journal of Computer Vision*, 129(11):3069–3087, 2021.
 - [72] Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye, and Dongwei Ren. Distance-iou loss: Faster and better learning for bounding box regression. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 12993–13000, 2020.
 - [73] Xingyi Zhou, Vladlen Koltun, and Philipp Krähenbühl. Tracking objects as points. In *ECCV*, pages 474–490. Springer, 2020.
 - [74] Xingyi Zhou, Dequan Wang, and Philipp Krähenbühl. Objects as points. *arXiv preprint arXiv:1904.07850*, 2019.
 - [75] Xingyi Zhou, Tianwei Yin, Vladlen Koltun, and Philipp Krähenbühl. Global tracking transformers. In *CVPR*, 2022.

A. Velocity Direction Variance in OCM

In this section, we work on the setting of linear motion with noisy states. We provide proof that the trajectory direction estimation has a smaller variance if the two states we use for the estimation have a larger time difference. We assume the motion model is $\mathbf{x}_t = f(t) + \epsilon$ where ϵ is gaussian noise and the ground-truth center position of the target is (μ_{u_t}, μ_{v_t}) at time step t . Then the true motion direction between the two time steps is

$$\theta = \arctan\left(\frac{\mu_{v_{t_1}} - \mu_{v_{t_2}}}{\mu_{u_{t_1}} - \mu_{u_{t_2}}}\right). \quad (9)$$

And we have $|\mu_{v_{t_1}} - \mu_{v_{t_2}}| \propto |t_1 - t_2|$, $|\mu_{u_{t_1}} - \mu_{u_{t_2}}| \propto |t_1 - t_2|$. As the detection results do not suffer from the error accumulation due to propagating along Markov process as Kalman filter does, we can assume the states from observation suffers some i.i.d. noise, i.e., $u_t \sim \mathcal{N}(\mu_{u_t}, \sigma_u^2)$ and $v_t \sim \mathcal{N}(\mu_{v_t}, \sigma_v^2)$. We now analyze the noise of the estimated $\tilde{\theta} = \frac{v_{t_1} - v_{t_2}}{u_{t_1} - u_{t_2}}$ by two observations on the trajectory. Because the function of $\arctan(\cdot)$ is monotone over the whole real field, we can study $\tan \theta$ instead which simplifies the analysis. We denote $w = u_{t_1} - u_{t_2}$, $y = v_{t_1} - v_{t_2}$, and $z = \frac{y}{w}$, first we can see that y and w jointly form a Gaussian distribution:

$$\begin{bmatrix} y \\ w \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu_y \\ \mu_w \end{bmatrix}, \begin{bmatrix} \sigma_y^2 & \rho\sigma_y\sigma_w \\ \rho\sigma_y\sigma_w & \sigma_w^2 \end{bmatrix}\right), \quad (10)$$

where $\mu_y = \mu_{v_{t_1}} - \mu_{v_{t_2}}$, $\mu_w = \mu_{u_{t_1}} - \mu_{u_{t_2}}$, $\sigma_w = \sqrt{2}\sigma_u$ and $\sigma_y = \sqrt{2}\sigma_v$, and ρ is the correlation coefficient between y and w . We can derive a closed-form solution of the probability density function [24] of z as

$$p(z) = \frac{g(z)e^{\frac{g(z)^2 - \alpha r(z)^2}{2\beta^2 r(z)^2}}}{\sqrt{2\pi}\sigma_w\sigma_y r(z)^3} \left[\Phi\left(\frac{g(z)}{\beta r(z)}\right) - \Phi\left(-\frac{g(z)}{\beta r(z)}\right) \right] + \frac{\beta e^{-2\alpha/\beta}}{\pi\sigma_w\sigma_y r(z)^2} \quad (11)$$

where

$$\begin{aligned} r(z) &= \sqrt{\frac{z^2}{\sigma_y^2} - \frac{2\rho z}{\sigma_y\sigma_w} + \frac{1}{\sigma_w^2}}, \\ g(z) &= \frac{\mu_y z}{\sigma_y^2} - \frac{\rho(\mu_y + \mu_w z)}{\sigma_y\sigma_w} + \frac{\mu_w}{\sigma_w^2}, \\ \alpha &= \frac{\mu_w^2 + \mu_y^2}{\sigma_y^2} - \frac{2\rho\mu_y\mu_w}{\sigma_w\sigma_y}, \quad \beta = \sqrt{1 - \rho^2}, \end{aligned} \quad (12)$$

and Φ is the cumulative distribution function of the standard normal. Without loss of generality, we can assume $\mu_w > 0$ and $\mu_y > 0$ because negative ground-truth displacements enjoy the same property. This solution has a good property

that larger μ_w or μ_y makes the probability density at the true value, i.e. $\mu_z = \frac{\mu_y}{\mu_w}$, higher, and the tails decay more rapidly. So the estimation of $\arctan \theta$, also θ , has smaller noise when μ_w or μ_y is larger. Under the assumption of linear motion, we thus should select two observations with a large temporal difference to estimate the direction.

It is reasonable to assume the noise of detection along the u-axis and v-axis are independent so $\rho = 0$. And when representing the center position in pixel, it is also moderate to assume $\sigma_w = \sigma_y = 1$ (also for the ease of presentation). Then, with different true value of $\mu_z = \frac{\mu_y}{\mu_w}$, the visualizations of $p(z)$ over z and μ_y are shown in Figure 5. The visualization demonstrates our analysis above. Moreover, it shows that when the value of μ_y or μ_w is small, the cluster peak of the distribution at μ_z is not significant anymore, as the noise σ_y and σ_w can be dominant. Considering the visualization shows that happens when μ_y is close to σ_y , this can happen when we estimate the speed by observations from two consecutive frames because the variance of observation can be close to the absolute displacement of object motion. This makes another support to our analysis in the main paper about the sensitivity to state estimation noise.

B. Interpolation by Gaussian Progress Regression

Interpolation as post-processing. Although we focus on developing an online tracking algorithm, we are also interested in whether post-process can further optimize the tracking results in diverse conditions. Despite the failure of GPR in online tracking in Table 6, we continue to study if GPR is better suited for interpolation in Table 8. We compare GPR with the widely-used linear interpolation. The maximum gap for interpolation is set as 20 frames and we use the same kernel for GPR as mentioned above. The results suggest that the GPR's non-linear interpolation is simply not efficient. We think this is due to limited data points which results in an inaccurate fit of the object trajectory. Further, the variance in regressor predictions introduces extra noise. Although GPR interpolation decreases the performance on MOT17-val significantly, its negative influence on DanceTrack is relatively minor where the object motion is more non-linear. We believe how to fit object trajectory with non-linear hypothesis still requires more study.

From the analysis in the main paper, the failure of SORT can mainly result from occlusion (lack of observations) or the non-linear motion of objects (the break of the linear-motion assumption). So the question arises naturally whether we can extend SORT free of the linear-motion assumption or at least more robust when it breaks.

One way is to extend from KF to non-linear filters, such as EKF [30, 52] and UKF [28]. However, for real-world online tracking, they can be hard to be adopted as they

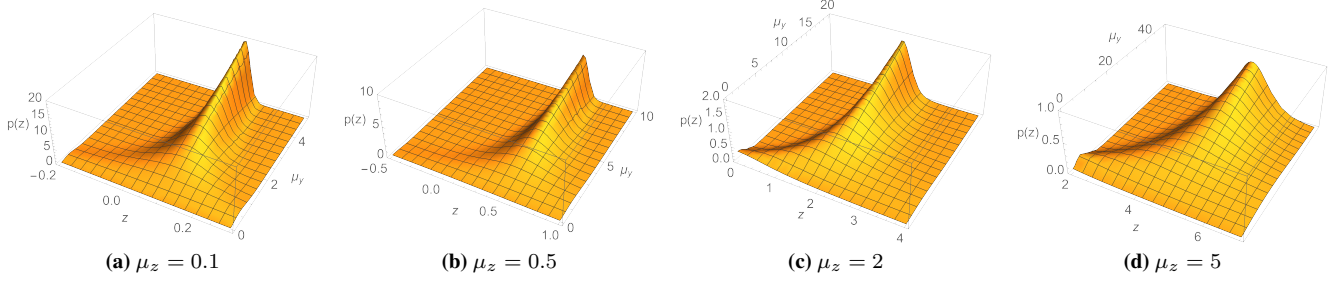


Figure 5. The probability density of $z = \tan \theta$ under different true value of z , i.e. $\mu_z = \frac{\mu_y}{\mu_w}$. We set μ_y and z as two variables. It shows that under different settings of true velocity direction when μ_y is smaller, the probability of estimated value with a significant shift from the true value is higher. As μ_y is proportional to the time difference of the two selected observations under linear motion assumption, it relates to the case that the two steps for velocity direction estimation has a shorter time difference.

Table 8. Ablation study about the interpolation post-processing.

	MOT17-val				DanceTrack-val			
	HOTA↑	AssA↑	MOTA↑	IDF1↑	HOTA↑	AssA↑	MOTA↑	IDF1↑
w/o interpolation	66.5	68.9	74.9	77.7	52.1	35.3	87.3	51.6
Linear Interpolation	68.0	69.9	77.9	79.3	52.8	35.6	89.8	52.1
GPR Interpolation	65.2	67.0	72.9	75.9	51.6	35.0	86.1	51.2

need knowledge about the motion pattern or still rely on the techniques fragile to non-linear patterns, such as linearization [29]. Another choice is to gain the knowledge beyond linearity by regressing previous trajectory, such as combining Gaussian Process (GP) [32, 47, 62]: given a observation $\mathbf{z}_*$ and a kernel function $k(\cdot, \cdot)$, GP defines gaussian functions with mean $\mu_{\mathbf{z}_*}$ and variance $\Sigma_{\mathbf{z}_*}$ as

$$\begin{aligned}\mu_{\mathbf{z}_*} &= \mathbf{k}_*^\top [\mathbf{K} + \sigma^2 \mathbf{I}]^{-1} \mathbf{y}, \\ \Sigma_{\mathbf{z}_*} &= k(\mathbf{z}_*, \mathbf{z}_*) - \mathbf{k}_*^\top [\mathbf{K} + \sigma^2 \mathbf{I}]^{-1} \mathbf{k}_*,\end{aligned}\quad (13)$$

where $\mathbf{k}_*$ is the kernel matrix between the input and training data and $\mathbf{K}$ is the kernel matrix over training data, $\mathbf{y}$ is the output of data. Until now, we have shown the primary study of using Gaussian Process Regression (GPR) in the online generation of the virtual trajectory in ORU and offline interpolation. But neither of them successfully boosts the tracking performance. Now, We continue to investigate in detail the chance of combining GPR and SORT for multi-object tracking for interpolation as some designs are worth more study.

B.1. Choice of Kernel Function in Gaussian Process

The kernel function is a key variable of GPR. There is not a generally efficient guideline to choose the kernel for Gaussian Process Regression though some basic observations are available [15]. When there is no additional knowledge about the time sequential data to fit, the RBF kernel is one of the most common choices:

$$k(\mathbf{x}, \mathbf{x}') = \sigma^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2l^2}\right), \quad (14)$$

where l is the lengthscale of the data to be fit. It determines the length of the “wiggles” of the target function. σ^2 is the output variance that determines the average distance of the function away from its mean. This is usually just a scale factor [15]. GPR is considered sensitive to l in some situations. So we conduct an ablation study over it in the offline interpolation to see if we can use GPR to outperform the linear interpolation widely used in multi-object tracking.

B.2. GPR for Offline Interpolation

We have presented the use of GPR in online virtual trajectory fitting and offline interpolation where we use $l^2 = 25$ and $\sigma = 1$ for the kernel in Eq. 14. Further, we make a more thorough study of the setting of GPR. We follow the settings of experiments in the main paper that only trajectories longer than 30 frames are put into interpolation. And the interpolation is only applied to the gap shorter than 20 frames. We conduct the experiments on the validation sets of MOT17 and DanceTrack.

For the value of l , we try fixed values, i.e. $l = 1$ and $l = 5$ ($2l^2 = 50$), value adaptive to trajectory length, i.e. $l = L_\tau$ and $l = 1000/L_\tau$, and the value output by Median Trick (MT) [18]. The training data is a series of quaternary $[u, v, w, h]$, normalized to zero-mean before being fed into training. The results are shown in Table 9. Linear interpolation is simple but builds a strong baseline as it can stably improve the tracking performance concerning multiple metrics. Directly using GPR to interpolate the missing points hurts the performance and the results of GPR are not sensitive to the setting of l .

Table 9. Ablation study about using Gaussian Process Regression for object trajectory interpolation. LI indicates Linear Interpolation, which is used to interpolate the trajectory before smoothing the trajectory by GPR. MT indicates Median Trick for kernel choice in regression. L_τ is the length of trajectory.

Interpolation Method	MOT17-val				DanceTrack-val			
	HOTA	AssA	MOTA	IDF1	HOTA	AssA	MOTA	IDF1
w/o interpolation	66.5	68.9	74.9	77.7	52.1	35.3	87.3	51.6
Linear Interpolation	69.6	69.9	77.9	79.3	52.8	35.6	89.8	52.1
GPR Interp, $l = 1$	66.2	67.6	74.3	76.6	51.8	35.0	86.6	50.8
GPR Interp, $l = 5$	66.3	67.0	72.9	75.9	51.8	35.1	86.5	51.1
GPR Interp, $l = L_\tau$	66.1	67.0	73.1	77.8	51.6	35.1	86.4	50.7
GPR Interp, $l = 1000/L_\tau$	65.9	67.0	73.0	77.8	51.8	35.0	86.9	51.0
GPR Interp, $l = \text{MT}(\tau)$	65.9	67.0	73.1	77.8	51.7	35.1	86.7	50.9
LI + GPR Smoothing, $l = 1$	69.5	69.6	77.8	79.3	52.8	35.6	89.9	52.1
LI + GPR Smoothing, $l = 5$	69.5	69.7	77.8	79.3	52.9	34.9	89.7	52.1
LI + GPR Smoothing, $l = L_\tau$	69.6	69.5	77.8	79.2	52.9	35.6	89.9	52.1
LI + GPR Smoothing, $l = 1000/L_\tau$	69.5	69.9	77.8	79.3	53.0	35.6	89.9	52.1
LI + GPR Smoothing, $l = \text{MT}(\tau)$	69.5	69.6	77.8	79.3	52.8	35.6	89.8	52.1

Table 10. Results on CroHD Head Tracking dataset [56]. Our method uses the detections from HeadHunter [56] or FairMOT [71] to generate new tracks.

Tracker	HOTA $\uparrow$	MOTA $\uparrow$	IDF1 $\uparrow$	FP(10^4) $\downarrow$	FN(10^4) $\downarrow$	IDs $\downarrow$	Frag $\downarrow$
HeadHunter [56]	36.8	57.8	53.9	5.18	30.0	4,394	15,146
HeadHunter dets + OC-SORT	39.0	60.0	56.8	5.18	28.1	4,122	10,483
FairMOT [71]	43.0	60.8	62.8	11.8	19.9	12,781	41,399
FairMOT dets + OC-SORT	44.1	67.9	62.9	10.2	16.4	4,243	10,122

Table 11. Results on DanceTrack test set. “Ours (MOT17)” uses the YOLOX detector trained on MOT17-training set.

Tracker	HOTA $\uparrow$	DetA $\uparrow$	AssA $\uparrow$	MOTA $\uparrow$	IDF1 $\uparrow$
SORT	47.9	72.0	31.2	91.8	50.8
OC-SORT	55.1	80.3	38.0	89.4	54.2
OC-SORT (MOT17)	48.6	71.0	33.3	84.2	51.5

There are two reasons preventing GPR from accurately interpolating missing segments. First, the trajectory is usually limited to at most hundreds of steps, providing very limited data points for GPR training to converge. On the other hand, the missing intermediate data points make the data series discontinuous, causing a huge challenge. We can fix the second issue by interpolating the trajectory with Linear Interpolation (LI) first and then smoothing the interpolated steps by GPR. This outperforms LI on DanceTrack but still regrades the performance by LI on MOT17. This is likely promoted by the non-linear motion on DanceTrack. By fixing the missing data issue of GPR, GPR can have a more accurate trajectory fitting over LI for the non-linear trajectory cases. But considering the outperforming from GPR is still minor compared with the Linear Interpolation-only version and GPR requires much heavier computation overhead, we do not recommend using such a practice in most multi-object tracking tasks. More careful and deeper study is still required on this problem.

C. Results on More Benchmarks

Results on HeadTrack [56]. When considering tracking in the crowd, focusing on only a part of the object can be beneficial [6] as it usually suffers less from occlusion than the full body. This line of study is conducted over hand tracking [40, 50], human pose [66] and head tracking [2, 43, 56] for a while. Moreover, with the knowledge of more fine-grained part trajectory, it can be useful in downstream tasks, such as action recognition [16, 17] and forecasting [7, 9, 31, 33]. As we are interested in the multi-object tracking in the crowd, we also evaluate the proposed OC-SORT on a recently proposed human head tracking dataset CroHD [56]. To make a fair comparison on only the association performance, we adopt OC-SORT by directing using the detections from existing tracking algorithms. The results are shown in Table 10. The detections of FairMOT [71] and HeadHunter [56] are extracted from their tracking results downloaded from the official leaderboard². We use the same parameters for OC-SORT as on the other datasets. The results suggest a significant tracking performance improvement compared with the previous methods [56, 71] for human body part tracking. But the tracking performance is still relatively low (HOTA= ~ 40). It is highly related to the difficulty of having accurate detection of tiny objects. Some samples from the test set of HeadTrack are shown in the first two rows of Figure 6.

²https://motchallenge.net/results/Head_Tracking_21/

Table 12. Results on MOT17 test set with the public detections. LI indicates Linear Interpolation.

Tracker	HOTA↑	MOTA↑	IDF1↑	FP(10^4)↓	FN(10^4)↓	IDs↓	Frag↓	AssA↑	AssR↑
CenterTrack [73]	-	61.5	59.6	1.41	20.1	2,583	-	-	-
QDTrack [42]	-	64.6	65.1	1.41	18.3	2,652	-	-	-
Lif.T [25]	51.3	60.5	65.6	1.50	20.7	1,189	3,476	54.7	59.0
TransCt [67]	51.4	68.8	61.4	2.29	14.9	4,102	8,468	47.7	52.8
TrackFormer [39]	-	62.5	60.7	3.28	17.5	2,540	-	-	-
OC-SORT	52.4	58.2	65.1	0.44	23.0	784	2,006	57.6	63.5
OC-SORT + LI	52.9	59.4	65.7	0.66	22.2	801	1,030	57.5	63.9

Table 13. Results on MOT20 test set with the public detections. LI indicates Linear Interpolation.

Tracker	HOTA↑	MOTA↑	IDF1↑	FP(10^4)↓	FN(10^4)↓	IDs↓	Frag↓	AssA↑	AssR↑
MPNTrack [4]	46.8	57.6	59.1	17.0	20.1	1,210	1,420	47.3	52.7
TransCt [67]	43.5	61.0	49.8	4.92	14.8	4,493	8,950	36.1	44.5
ApLift [26]	46.6	58.9	56.5	1.77	19.3	2,241	2,112	45.2	48.1
TMOH [53]	48.9	60.1	61.2	3.80	16.6	2,342	4,320	48.4	52.9
LPC_MOT [13]	49.0	56.3	62.5	1.17	21.3	1,562	1,865	52.4	54.7
OC-SORT	54.3	59.9	67.0	0.44	20.2	554	2,345	59.5	65.1
OC-SORT + LI	55.2	61.7	67.9	0.57	19.2	508	805	59.8	65.9

Public Tracking on MOT17 and MOT20. Although we use the same object detectors as some selected baselines, there is still variances in detections when compared with other methods. Therefore, we also report with the public detections on MOT17/MOT20 in Table 12 and Table 13. OC-SORT still outperforms the existing state-of-the-arts in the public tracking setting. And the outperforming of OC-SORT is more significant on MOT20 which has more severe occlusion scenes. Some samples from the test set of MOT20 are shown in the last row in Figure 6.

D. Pseudo-code of OC-SORT

See the pseudo-code of OC-SORT in Algorithm. 1.

E. More Results on DanceTrack

To gain more intuition about the improvement of OC-SORT over SORT, we provide more comparisons. In Figure 8, we show more samples where SORT suffers from ID switch or Fragmentation caused by non-linear motion or occlusion but OC-SORT survives. Furthermore, in Figure 9, we show more samples of trajectory visualizations from SORT and OC-SORT on DanceTrack-val set.

DanceTrack [54] is proposed to encourage better association algorithms instead of carefully tuning detectors. We train YOLOX [19] detector on MOT17 training set only to provide detections on DanceTrack. We find the tracking performance of OC-SORT is already higher than the baselines (Table 11). We believe the potential to improve multi-object tracking by better association strategy is still promising and DanceTrack is a good platform for the evaluation.

F. Integrate Appearance into OC-SORT

OC-SORT is pure motion-based but flexible to integrate with other association cues, such as object appearance. We

make an attempt of adding appearance information into OC-SORT and achieve significant performance improvements, validated by experiments on MOT17, MOT20, and DanceTrack. Please refer to Deep OC-SORT [38] for details.

G. More Discussion of State Noise Sensitivity

In Section 3.2.1, we show that the noise of state estimate will be amplified to the noise of velocity estimate. This is because the velocity estimate is correlated to the state estimate. But the analysis is in a simplified model in which velocity itself does not gain noise from the transition directly and the noise of state estimate is i.i.d on different steps. However, in the general case, such a simplification does not hold. We now provide a more general analysis of the state noise sensitivity of SORT.

For the process in Eq 1, we follow the most commonly adapted implementation of Kalman filter ³ and SORT ⁴ for video multi-object tracking. Instead of writing the mean state estimate, we consider the noisy prediction of state estimate now, which is formulated as

$$\mathbf{x}_{t|t-1} = \mathbf{F}_t \mathbf{x}_{t-1} + \mathbf{w}_t, \quad (15)$$

where $\mathbf{w}_t$ is the process noise, drawn from a zero mean multivariate normal distribution, $\mathcal{N}$, with covariance, $\mathbf{w}_t \sim \mathcal{N}(0, \mathbf{Q}_t)$. As $\mathbf{x}_t$ is a seven-tuple, *i.e.* $\mathbf{x}_t = [u, v, s, r, \dot{u}, \dot{v}, \dot{s}]^\top$, the process noise applies to not just the state estimate but also the velocity estimates. Therefore, for a general form of analysis of temporal error magnification in Eq 5, we would get a different result because not just the position terms but also the velocity terms gain noise from the transition process. And the noise of velocity terms will amplify the noise of position estimate by the transition at

³<https://github.com/rlabbe/filterpy>

⁴<https://github.com/abewley/sort>

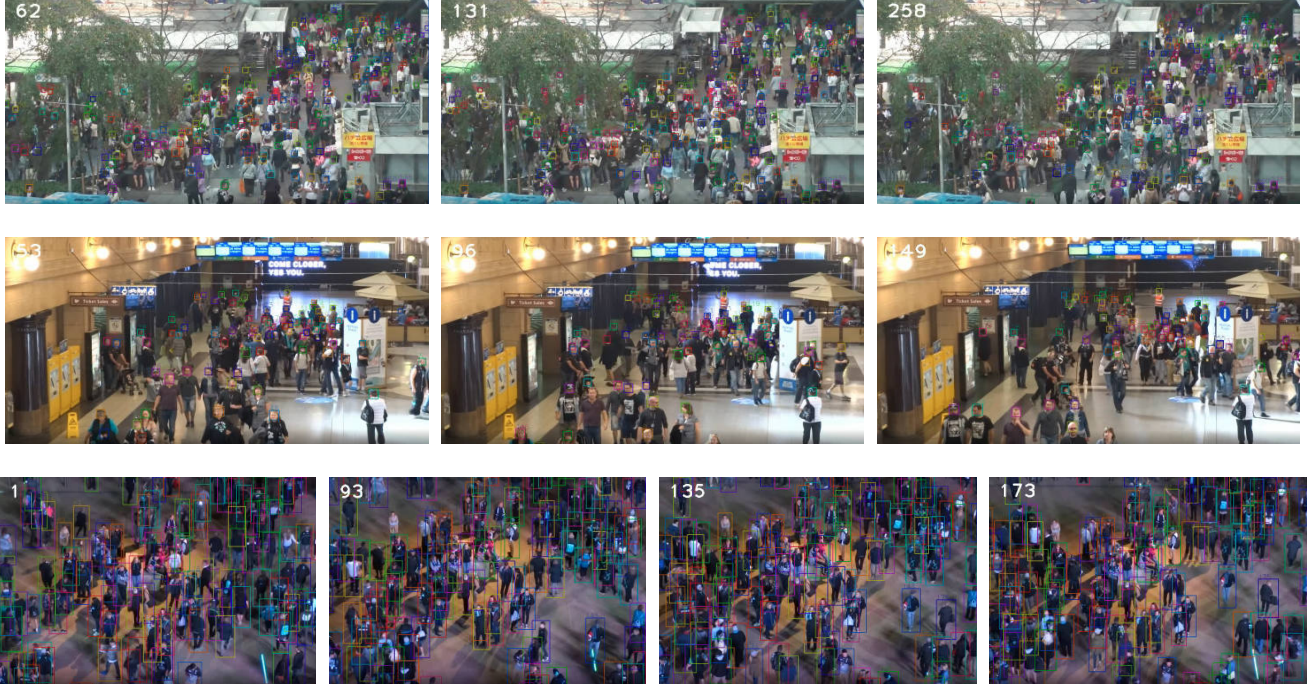


Figure 6. The visualization of the output of OC-SORT on randomly selected samples from the test set of HeadTrack [56] (the first two rows) and MOT20 [14] (the bottom row). These two datasets are both challenging because of the crowded scenes where pedestrians have heavy occlusion with each other. OC-SORT achieves superior performance on both datasets.

the next step. We note the process noise as in practice:

$$\mathbf{Q}_t = \begin{bmatrix} \sigma_u^2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & \sigma_v^2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \sigma_s^2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \sigma_r^2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \sigma_u^2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \sigma_v^2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \sigma_s^2 \end{bmatrix}, \quad (16)$$

and the linear transition model as

$$\mathbf{F}_t = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}. \quad (17)$$

We assume the time step when a track gets untracked is t_1 and don't consider the noise from previous steps. For simplicity, we assume the motion in the x-direction and y-direction do not correlate. We take the motion on the x-direction as an example without loss of generality:

$$\delta_{u_{t_0}} \sim \mathcal{N}(0, \sigma_u^2), \quad \delta_{\dot{u}_{t_0}} \sim \mathcal{N}(0, \sigma_{\dot{u}}^2). \quad (18)$$

On the next step, with no correction from the observation, the error would be accumulated ($\Delta t = 1$),

$$\delta_{u_{t_0+1}} \sim \mathcal{N}(0, 2\sigma_u^2 + \sigma_{\dot{u}}^2), \quad \delta_{\dot{u}_{t_0+1}} \sim \mathcal{N}(0, 2\sigma_{\dot{u}}^2). \quad (19)$$

Therefore, the accumulation is even faster than we analyze in Section 3.2 as

$$\delta_{u_{t_0+T}} \sim \mathcal{N}(0, (T+1)\sigma_u^2 + \frac{1}{2}T(T+1)\sigma_{\dot{u}}^2). \quad (20)$$

In the practice of SORT, we have to suppress the noise from velocity terms because it is too sensitive. We achieve it by setting a proper value for the process noise $\mathbf{Q}_t$. For example, the most commonly adopted value⁵ of $\mathbf{Q}_t$ in SORT is

$$\mathbf{Q}_t = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.01 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.01 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.0001 \end{bmatrix}. \quad (21)$$

In such a parameter setting, we have the ratio between

⁵<https://github.com/abewley/sort/blob/master/sort.py#L111>

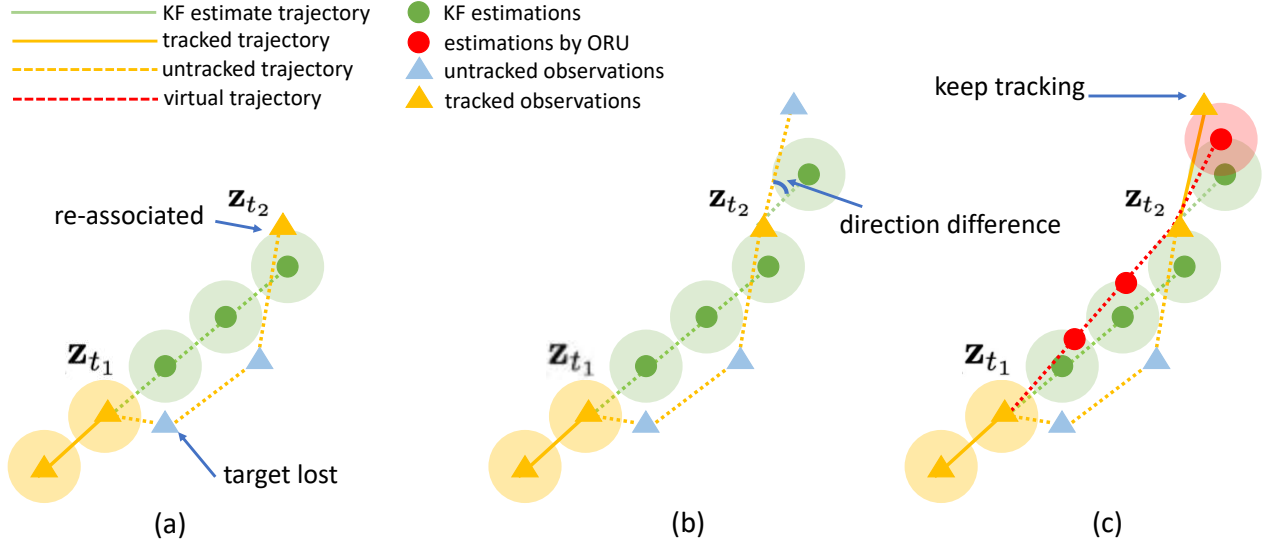


Figure 7. Illustration of how ORU changes the behaviors of SORT after an untracked track is re-associated to an observation. The circle area with shadow indicates the range that an estimate can be associated with observations close enough to it. (a). The track is re-associates with an observation z_{t_2} at the step t_2 after being untracked since the time step t_1 . (b). Without ORU, on the next step of re-association, even though the KF state is updated by z_{t_2} , there is still a direction difference between the true object trajectory and the KF estimates. Therefore, the track is unmatched with detections again (in blue). (c). With ORU, we get a more significant change in the state, especially the motion direction by updating velocity. Now, the state estimate (in red) is closer to the state observation and they can be associated again.

the noise from position terms and velocity terms as

$$\beta = \frac{(T+1)\sigma_u^2}{0.5T(T+1)\sigma_u^2} = \frac{200}{T}. \quad (22)$$

In practice, a track is typically deleted if it keeps untracked for T_{del} time steps. Usually we set $T_{\text{del}} < 10$, so we have $\beta > 20$. Therefore, we usually consider the noise from velocity terms as secondary. Such a convention allows us to use the simplified model in Section 3.2.1 for noise analysis. But it also brings a side-effect that SORT can't allow the velocity direction of a track to change quickly in a short time interval. We will see later (Section H) that it makes trouble to SORT when non-linear motion and occlusion come together and motivates the design of ORU in OC-SORT.

H. Intuition behind ORU

ORU is designed to fix the error accumulated during occlusion when an untracked track is re-associated with an observation. But in general, the bias in the state estimate $\hat{x}$ after being untracked for T time steps can be fixed by the update stage once it gets re-associated with an observation. To be precise, the Optimal Kalman gain, *i.e.* K_t , can use the re-associated observation to update the KF posteriori parameters. In general, such an expectation of KF's behavior is reasonable. But because we usually set the corresponding covariance for velocity terms very small (Eq 21), it is

difficult for SORT to steer to the correct velocity direction at the step of re-association.

Motivated by such observations, we design ORU. In the simplified model shown in Figure 7, the circle area with the shadow around each estimate footage is the eligible range to associate with observations inside. ORU is designed for the case that a track is re-associated after being untracked. Therefore, the typical situation is as shown in the figure that the true trajectory first goes away from the linear trajectory of KF estimates and then goes closer to it so that there can be a re-association. After the re-association, there would be a cross of the two trajectories.

In SORT, after re-associating with an observation, the direction of the velocity of the previously untracked track still has a significant difference from the true value. This is shown in Figure 7(b). This makes the estimate on the future steps lost again (the blue triangle). The reason is the convention of Q discussed in Appendix G. Therefore, even though the canonical KF can support fixing the accumulated error during being untracked theoretically, it is very rare in practice. In ORU, we follow the virtual trajectory where we have multiple virtual observations. In this way, even if the value of $Q[4:, 4:]$ is small, we can still have a better-calibrated velocity direction after the time step t_2 . We would like to note that the intuition behind ORU is from our observations in practice and based on the common convention of using Kalman filter for multi-object tracking. It

does not make fundamental changes to upgrade the power of the canonical Kalman filter.

Here we provide a more formal mathematical expression to compare the behaviors of SORT and OC-SORT. Assume that the track was lost at the time step t_1 and re-associated at t_2 . We assume the mean state estimate is

$$\hat{\mathbf{x}}_{t_1|t_1} = [u_1, v_1, s_1, r_1, \dot{u}_1, \dot{v}_1, \dot{s}_1]^\top, \quad (23)$$

and the covariance at t_1 is

$$\mathbf{P}_{t_1|t_1} = \begin{bmatrix} \sigma_{u_1}^2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & \sigma_{v_1}^2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \sigma_{s_1}^2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \sigma_{r_1}^2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \sigma_{\dot{u}_1}^2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \sigma_{\dot{v}_1}^2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \sigma_{\dot{s}_1}^2 \end{bmatrix}. \quad (24)$$

Then, because the covariance expands from the input of process noise at each step of *predict*, at t_2 , we have the priori estimates ($t_\Delta = t_2 - t_1$) of state

$$\hat{\mathbf{x}}_{t_2|t_2-1} = [u_2, v_2, s_2, r_2, \dot{u}_2, \dot{v}_2, \dot{s}_2]^\top, \quad (25)$$

with

$$\begin{aligned} u_2 &= u_1 + \dot{u}_1 t_\Delta, \\ v_2 &= v_1 + \dot{v}_1 t_\Delta, \\ s_2 &= s_1 + \dot{s}_1 t_\Delta, \\ r_2 &= r_1, \\ \dot{u}_2 &= \dot{u}_1, \\ \dot{v}_2 &= \dot{v}_1, \\ \dot{s}_2 &= \dot{s}_1. \end{aligned} \quad (26)$$

And the priori covariance

$$\mathbf{P}_{t_2|t_2-1} = \begin{bmatrix} \sigma_{u_2}^2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & \sigma_{v_2}^2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \sigma_{s_2}^2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \sigma_{r_2}^2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \sigma_{\dot{u}_2}^2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \sigma_{\dot{v}_2}^2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \sigma_{\dot{s}_2}^2 \end{bmatrix}, \quad (27)$$

with

$$\begin{aligned} \sigma_{u_2}^2 &= \sigma_{u_1}^2 + t_\Delta(\sigma_u^2 + \sigma_{\dot{u}_1}^2) + \frac{t_\Delta(t_\Delta - 1)}{2} \sigma_{\ddot{u}}^2, \\ \sigma_{v_2}^2 &= \sigma_{v_1}^2 + t_\Delta(\sigma_v^2 + \sigma_{\dot{v}_1}^2) + \frac{t_\Delta(t_\Delta - 1)}{2} \sigma_{\ddot{v}}^2, \\ \sigma_{s_2}^2 &= \sigma_{s_1}^2 + t_\Delta(\sigma_s^2 + \sigma_{\dot{s}_1}^2) + \frac{t_\Delta(t_\Delta - 1)}{2} \sigma_{\ddot{s}}^2, \\ \sigma_{r_2}^2 &= \sigma_{r_1}^2 + t_\Delta \sigma_r^2, \\ \sigma_{\dot{u}_2}^2 &= \sigma_{\dot{u}_1}^2 + t_\Delta \sigma_{\ddot{u}}^2, \\ \sigma_{\dot{v}_2}^2 &= \sigma_{\dot{v}_1}^2 + t_\Delta \sigma_{\ddot{v}}^2, \\ \sigma_{\dot{s}_2}^2 &= \sigma_{\dot{s}_1}^2 + t_\Delta \sigma_{\ddot{s}}^2. \end{aligned} \quad (28)$$

Now, SORT will keep going forward as normal. Therefore, with the re-associated observation $\mathbf{z}_{t_2}$, we have

$$\text{SORT} \begin{cases} \hat{\mathbf{x}}_{t_2|t_2} = \hat{\mathbf{x}}_{t_2|t_2-1} + \mathbf{K}_{t_2}(\mathbf{z}_{t_2} - \mathbf{H}\hat{\mathbf{x}}_{t_2|t_2-1}), \\ \mathbf{P}_{t_2|t_2} = (\mathbf{I} - \mathbf{K}_{t_2}\mathbf{H})\mathbf{P}_{t_2|t_2-1} \end{cases} \quad (29)$$

where the observation model is

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}, \quad (30)$$

and the Kalman gain is

$$\mathbf{K}_{t_2} = \mathbf{P}_{t_2|t_2-1}\mathbf{H}^\top(\mathbf{H}\mathbf{P}_{t_2|t_2-1}\mathbf{H}^\top + \mathbf{R}_{t_2})^{-1}. \quad (31)$$

On the other hand, OC-SORT will replay Kalman filter *predict* on a generated virtual trajectory to gain the posteriori estimates on t_2 (ORU). With the default linear motion analysis, we have the virtual trajectory as

$$\tilde{\mathbf{z}}_t = \mathbf{z}_{t_1} + \frac{t - t_1}{t_2 - t_1}(\mathbf{z}_{t_2} - \mathbf{z}_{t_1}), t_1 < t < t_2. \quad (32)$$

Now, to derive the posteriori estimate, we will run the loop between *predict* and *re-update* from t_1 to t_2 .

$$\text{OC-SORT} \begin{cases} \hat{\mathbf{x}}_{t|t} = \mathbf{F}\hat{\mathbf{x}}_{t-1|t-1} + \mathbf{K}_t(\tilde{\mathbf{z}}_t - \mathbf{H}\mathbf{F}\hat{\mathbf{x}}_{t-1|t-1}) \\ \mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t\mathbf{H})(\mathbf{F}\mathbf{P}_{t-1|t-1}\mathbf{F}^\top + \mathbf{Q}_t) \end{cases} \quad (33)$$

where the Kalman gain is

$$\mathbf{K}_t = \mathbf{P}_{t|t-1}\mathbf{H}_t^\top(\mathbf{H}_t\mathbf{P}_{t|t-1}\mathbf{H}_t^\top + \mathbf{R}_t)^{-1}, \quad (34)$$

and we can always rewrite it with

$$\mathbf{P}_{t|t-1} = \mathbf{F}\mathbf{P}_{t-1|t-1}\mathbf{F}^\top + \mathbf{Q}_t. \quad (35)$$

In the common practice of Kalman filter, we assume a constant set of Gaussian noise for the process noise $\mathbf{Q}_t$. This assumption typically can't hold in practice. This makes the conflict that when there are consistent observations over time, we require a small process noise for multi-object tracking in high-frame-rate videos. However, when there is a period of observation missing, the direction difference between the true direction and the direction maintained by the linear motion assumption grows. This causes the failure of SORT to consistently track previously lost targets even after re-association.

We show the different outcomes of SORT and OC-SORT upon re-associating lost targets in Eq 29 and Eq 33. Analyzing their difference more deeply will require more assumptions of the underlying true object trajectory and the observations. Therefore, instead of theoretical proof, we demonstrate the gain of performance from OC-SORT over SORT empirically as shown in the experiments.

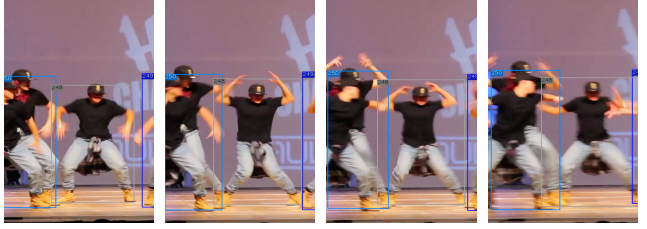
Algorithm 1: Pseudo-code of OCSORT.

Input: Detections $\mathcal{Z} = \{\mathbf{z}_k^i | 1 \leq k \leq T, 1 \leq i \leq N_k\}$; Kalman Filter KF; threshold to remove untracked tracks t_{expire}
Output: The set of tracks $\mathcal{T} = \{\tau_i\}$

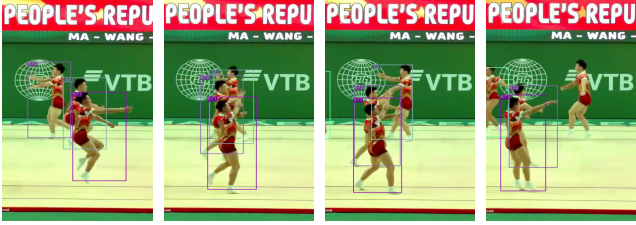
```
1 Initialization:  $\mathcal{T} \leftarrow \emptyset$  and KF;  
2 for timestep  $t \leftarrow 1 : T$  do  
    /* Step 1: match track prediction with observations */  
3     $\mathbf{Z}_t \leftarrow [\mathbf{z}_t^1, \dots, \mathbf{z}_t^{N_t}]^\top$  /* Observations */  
4     $\hat{\mathbf{X}}_t \leftarrow [\hat{\mathbf{x}}_t^1, \dots, \hat{\mathbf{x}}_t^{|\mathcal{T}|}]^\top$  from  $\mathcal{T}$  /* Estimations by KF.predict */  
5     $\mathcal{Z} \leftarrow$  Historical observations on the existing tracks  
6     $C_t \leftarrow C_{\text{IoU}}(\hat{\mathbf{X}}_t, \mathbf{Z}_t) + \lambda C_v(\mathcal{Z}, \mathbf{Z}_t)$  /* Cost Matrix with OCM term */  
7    Linear assignment by Hungarians with cost  $C_t$   
8     $\mathcal{T}_t^{\text{matched}} \leftarrow$  tracks matched to an observation  
9     $\mathcal{T}_t^{\text{remain}} \leftarrow$  tracks not matched to any observation  
10    $\mathbf{Z}_t^{\text{remain}} \leftarrow$  observations not matched to any track  
  
    /* Step 2: perform OCR to find lost tracks back */  
11    $\mathbf{Z}^{\mathcal{T}_t^{\text{remain}}} \leftarrow$  last matched observations of tracks in  $\mathcal{T}_t^{\text{remain}}$   
12    $C_t^{\text{remain}} \leftarrow C_{\text{IoU}}(\mathbf{Z}^{\mathcal{T}_t^{\text{remain}}}, \mathbf{Z}_t^{\text{remain}})$   
13   Linear assignment by Hungarians with cost  $C_t^{\text{remain}}$   
14    $\mathcal{T}_t^{\text{recovery}} \leftarrow$  tracks from  $\mathcal{T}_t^{\text{remain}}$  and matched to observations in  $\mathbf{Z}^{\mathcal{T}_t^{\text{remain}}}$   
15    $\mathbf{Z}_t^{\text{unmatched}} \leftarrow$  observations from  $\mathbf{Z}^{\mathcal{T}_t^{\text{remain}}}$  that are still unmatched to tracks  
16    $\mathcal{T}_t^{\text{unmatched}} \leftarrow$  tracks from  $\mathcal{T}_t^{\text{remain}}$  that are still unmatched to observations  
17    $\mathcal{T}_t^{\text{matched}} \leftarrow \{\mathcal{T}_t^{\text{matched}}, \mathcal{T}_t^{\text{recovery}}\}$   
  
    /* Step 3: update status of matched tracks */  
18   for  $\tau$  in  $\mathcal{T}_t^{\text{matched}}$  do  
19       if  $\tau.\text{tracked} = \text{False}$  then  
20           /* Perform ORU for track from untracked to tracked */  
21            $\mathbf{z}_{t'}^\tau, t' \leftarrow$  The last observation matched to  $\tau$  and the time step  
22           Rollback KF parameters to  $t'$   
23           /* Generate virtual observation trajectory */  
24            $\hat{\mathbf{Z}}_t^\tau \leftarrow [\hat{\mathbf{z}}_{t'+1}^\tau, \dots, \hat{\mathbf{z}}_{t-1}^\tau]$   
25           Online smooth KF parameters along  $\hat{\mathbf{Z}}_t^\tau$   
26       end  
27        $\tau.\text{tracked} = \text{True}$   
28        $\tau.\text{untracked} = 0$   
29       Append the new matched associated observation  $\mathbf{z}_t^\tau$  to  $\tau$ 's observation history  
30       Update KF parameters for  $\tau$  by  $\mathbf{z}_t^\tau$   
31   end  
  
    /* Step 4: initialize new tracks and remove expired tracks */  
32    $\mathcal{T}_t^{\text{new}} \leftarrow$  new tracks generated from  $\mathbf{Z}_t^{\text{unmatched}}$   
33   for  $\tau$  in  $\mathcal{T}_t^{\text{unmatched}}$  do  
34        $\tau.\text{tracked} = \text{False}$   
35        $\tau.\text{untracked} = \tau.\text{untracked} + 1$   
36   end  
37    $\mathcal{T}_t^{\text{reserved}} \leftarrow \{\tau | \tau \in \mathcal{T}_t^{\text{unmatched}} \text{ and } \tau.\text{untracked} < t_{\text{expire}}\}$  /* remove expired unmatched tracks */  
38    $\mathcal{T} \leftarrow \{\mathcal{T}_t^{\text{new}}, \mathcal{T}_t^{\text{matched}}, \mathcal{T}_t^{\text{reserved}}\}$  /* Conclude */  
39 end  
40  $\mathcal{T} \leftarrow \text{Postprocess}(\mathcal{T})$  /* [Optional] offline post-processing */  
41 Return:  $\mathcal{T}$ 
```



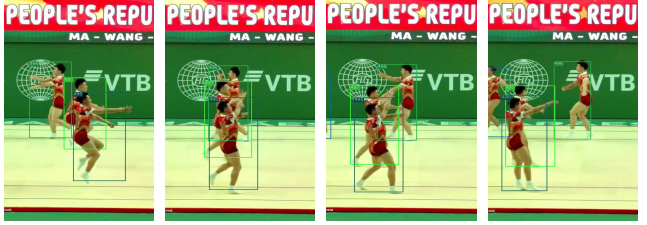
(a) SORT: dancetrack0036



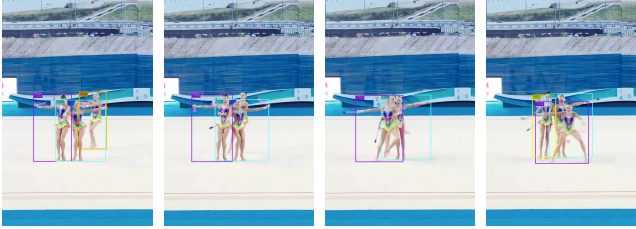
(b) OC-SORT: dancetrack0036



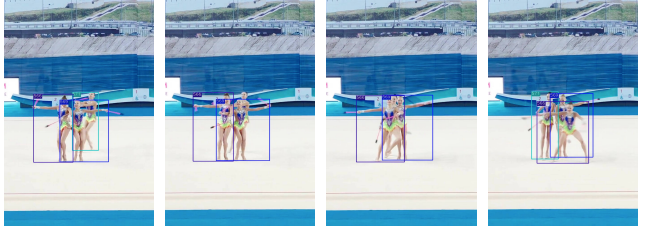
(c) SORT: dancetrack0054



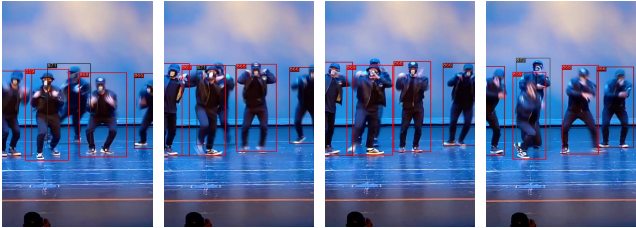
(d) OC-SORT: dancetrack0054



(e) SORT: dancetrack0064



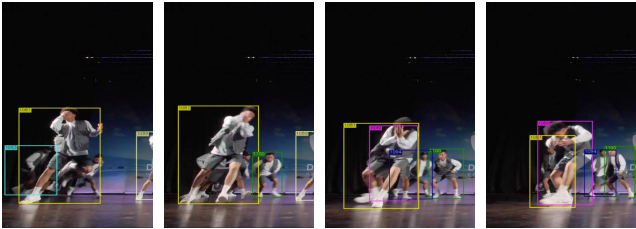
(f) OC-SORT: dancetrack0064



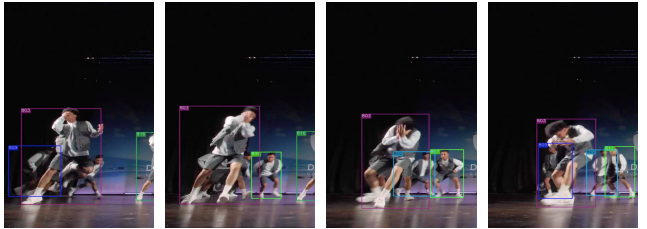
(g) SORT: dancetrack0078



(h) OC-SORT: dancetrack0078



(i) SORT: dancetrack0089



(j) OC-SORT: dancetrack0089



(k) SORT: dancetrack0100



(l) OC-SORT: dancetrack0100

Figure 8. More samples where SORT suffers from the fragmentation and ID switch of tracks from occlusion or non-linear motion but OC-SORT survives. To be precise, the issue happens on the objects by SORT at: (a) #322 $\rightarrow$ #324; (c) ID switch between #672 and #673, later #673 being lost; (e) #760 $\rightarrow$ #761; (g) #871 $\rightarrow$ #872; (i) #1063 $\rightarrow$ #1090, then ID switch with #1081; (l) #1295 $\rightarrow$ #1304. We select samples from diverse scenes, including street dance, classic dance and gymnastics. Best viewed in color and zoomed in.

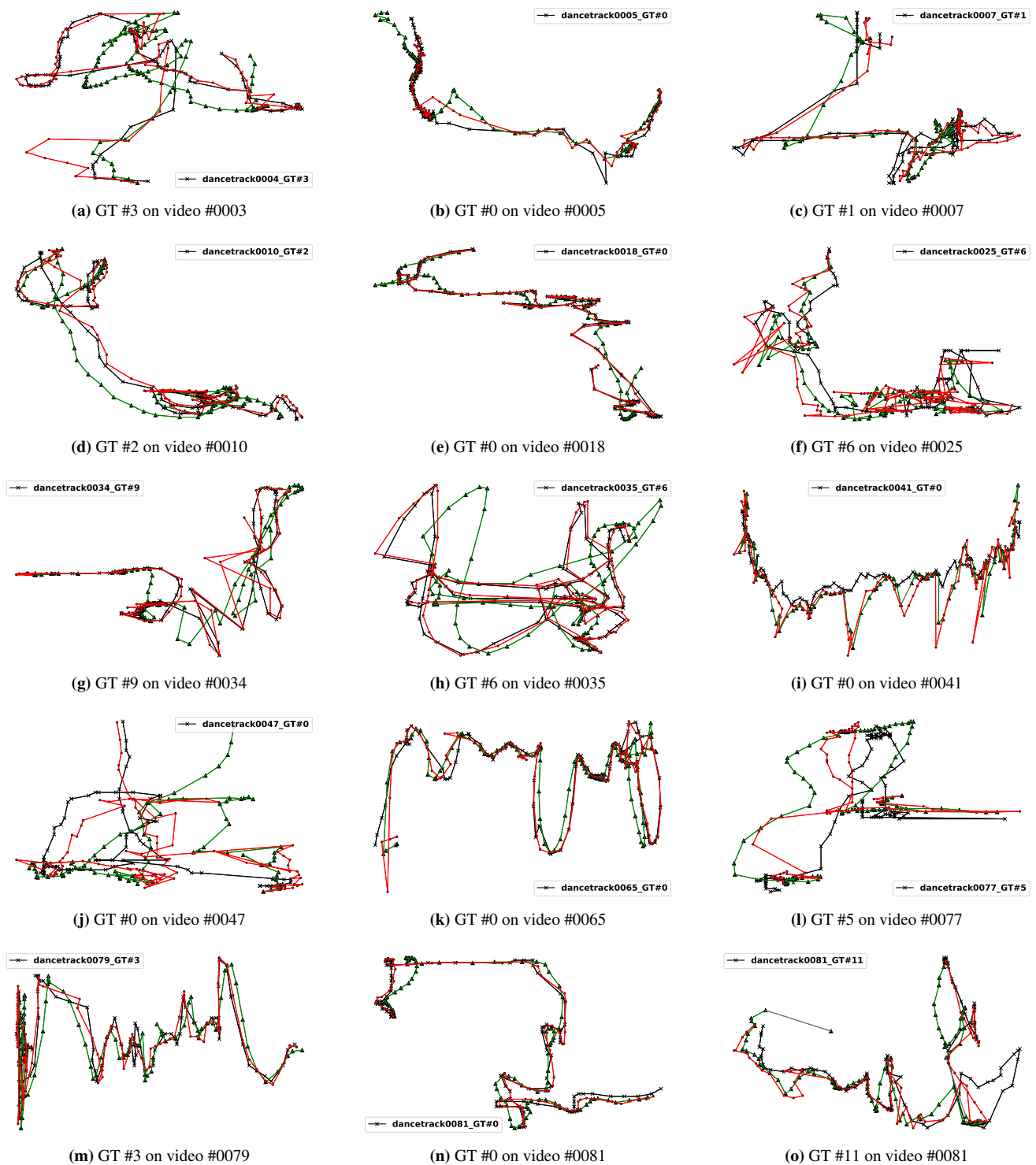


Figure 9. Randomly selected object trajectories on the videos from DanceTrack-val set. The **black cross** indicates the ground truth trajectory. The **red dots** indicate the trajectory output by OC-SORT and associated to the selected GT trajectory. The **green triangles** indicate the trajectory output by SORT and associated to the selected GT trajectory. SORT and OC-SORT use the same hyperparameters and detections. Trajectories are sampled at the first 100 frames of each video sequence.